



ONE IMAGE, END TO END

A Poster, Start to Finish

One Image from Blank Project to Finished Print



PLAKAT
ARTISTIC CREATION TOOL

Vladimir Ulogov

2026 · examples assume plakat 6.30.0 or newer

Contents

The poster we are making	3
The one idea to take away	4
The shape of the journey	5
What you need	5
Part I — Setting Up	7
1 — Installing plakat, and Your First Image	8
Getting plakat	9
Checking the machine: `doctor`	11
Choosing the first model	12
The first image	13
A note on speed and patience	15
2 — The Prose Project	17
Bootstrapping with `init`	18
The file you actually write: `prompts.txt`	19
What compile does — the short version	20
Why a whole workflow for one picture?	21
Part II — Authoring in Prose	23
3 — Writing the Scene in Prose	24
Sentences, not keyword soup	25
Naming the pieces: components	26
Leaning on a word: weighting	30
Saying what you don't want: negatives	31
Where the scene stands	32
4 — Will It Even Render?	34
The feasibility report: `compile --analyze`	35
Why this is worth a whole command	37
The polish loop	38
Where analyze fits the arc	39
5 — Fixing the Prose — and Remembering Why	40
Applying the safe fixes: `--analyze --fix`	41
Remembering why: the smysl corpus	42
Asking the corpus: `compile --trace`	44
Where the loop has taken us	44
Part III — Compiling & Running	46
6 — Compiling to a Scenario	47

The basic compile	48
The enhancer, and choosing a provider	48
Seeing what it will do: <code>--explain`</code>	51
Splitting the prose: <code>@include`</code>	52
The scenario it produced	52
7 — Running It, and Choosing a Model	55
Running the scenario	55
The model families	57
The token budget follows the model	58
Sizes and the native-resolution rule	59
What will run here?	59
The poster's model	60
8 — Tuning the Image	62
Where the dials live: the scenario	63
The scratchpad: scouting with <code>generate`</code>	64
Letting plakats find the better prompt: <code>compile --improve`</code>	66
Tuning that lives in the prose	67
The dial you don't turn: fitting the budget	68
Part IV — Composition & Control	71
9 — More Than One Figure	72
The control ladder	73
Naming the figures: <code>foreground:`</code>	73
Turning on composition: <code>control-generate`</code>	74
Posing the figures	75
The regional toggle	76
Where the poster stands	77
10 — A Cart, and a Relationship	79
The fusion problem, concretely	80
Declaring a separate object: <code>objects:`</code>	80
Relating the figures: <code>relate:`</code>	82
An honest word on the ceiling	83
11 — A Look, and a Face That Stays	85
Reaching for a style	86
Naming a LoRA — and its trigger	88
Two other roads to a style	88
A face that stays: personas	89
The poster, dressed	90
Part V — Editing & Finishing	92

12 — Editing What You Have	93
Changing the whole image a little: <code>`img2img`</code>	94
Fixing one spot: masks and inpaint	94
Removing something entirely: <code>`remove`</code>	95
Swapping the background: <code>`replace-bg`</code>	95
Faces: <code>`restore-faces`</code> and <code>`faceswap`</code>	96
Extending the frame: <code>`outpaint`</code>	97
Edit, don't re-roll	97
13 — Making It Look Real	99
The AI fingerprint, and how to dull it	100
Realism, not a filter	101
Protecting the figures	102
Naturalize from the prose	103
A correct image becomes a finished one	103
14 — Bigger, and Sharper	105
The fast road: classical upscale	106
The detailed road: diffusion upscale	107
Faces survive the enlargement	108
How big is big enough?	109
A poster at print size	109
15 — The Title, the Signature, and the Export	111
The title: type belongs in a layout, not a prompt	112
Signing the work: provenance etching	114
The recipe travels with the image	114
Choosing the file	115
Start to finish	115
16 — Layered Generation	117
The idea: constraints, not pixels	118
The plan	118
Size classes: what the anchor can hold	120
Rendering	121
Cohesion: the anchor is a dial	122
Structure and self-correction: <code>--depth-control</code> and <code>--adapt</code>	124
Did it land? Verify, repair, lift	125
Skipping the hand-written plan: the planner	126
Layering from prose: <code>compile --layered</code>	126
Which road? Layered vs. the compile path	128
Part VI — Reference	130

Appendix A — The Prose Directive Reference	131
Project & output	132
Sampler & quality	132
Reuse & structure	132
Composition & control	133
Finishing	134
In-prose syntax	134
The commands that act on prose	134
Appendix B — Models, Families, and Budgets	136
The families at a glance	137
Common aliases	137
The budget, keyed to the model	138
Native sizes	138
Choosing, in one breath	138
Appendix C — The Polish Loop, on One Page	140
The loop, step by step	141
The commands, in order	141
Why analyze first	142
smysl: the memory of the loop	142
Reading a feasibility report	143
The control ladder, when analyze says climb	143
Where to go next	144
Colophon	145
About the Author	146
A work of love	147
A note on cooperation	147
Where to find more	148

Why plakat?

Most image tools ask you to do the same thing: type a wish into a box, wait, and hope. When the picture comes back wrong — a fused figure, a garbled hand, a scene that was impossible from the first word — you change a word and hope again. The compute is spent, the reasoning is gone, and nothing you learned survives to the next attempt. It is a slot machine with a progress bar.

plakat is built on a different bet: that making a good image is **work you can reason about**, not luck you wait on. It is a local, pure-Rust studio — every image is rendered on your own machine, it phones nothing home, and it asks for no subscription. Its optional steps that **reason about** your prose — enhancing a prompt, judging a render, improving a scene — run on a small on-device model by default, so nothing need ever leave your computer. The one time anything does is if **you** choose to route those steps through a hosted language model; even then you can stay fully local with a model run under Ollama. That choice is always yours, and Chapter 6 spells out exactly when text leaves the machine and how to keep it from doing so. But its real idea is smaller and more useful than any feature list. You write your scene in **prose**, in a plain file you own. Before you spend a single render, plakats reads that prose and tells you what will probably fail — and why. You fix the prose, cheaply, and it remembers the reason for every change. Only when the scene is sound do you commit the compute. The slot machine becomes a workbench.

That is the whole reason I built it. Not to have the most models or the loudest demos — it has plenty of both — but to respect the two things a maker cannot get back: the **time** a wrong render burns, and the **thread** of why a picture turned out the way it did. A tool that tells you the scene is over-stuffed before you wait three minutes to discover it, and that can answer “why is this phrase in my prompt?” a month later, is a tool that treats your attention as worth something.

This book is the argument made concrete. Rather than catalogue what every command does, it follows one image — a small night-market poster — from an empty project to a signed, printable file, reaching for each tool exactly when the work asks for it. By the end you will not merely know the flags; you will know the **order**, which is the part no reference page can teach. If you have ever felt that image models make you their operator instead of the other way around, this is the book — and plakats is the tool — that hands the controls back.

Following One Poster



NIGHT MARKET — the poster this book builds, one command at a time.

plakat is a large program. It generates images from text, transfers styles, places synthetic people, swaps faces, builds comics and textures and maps and fractals, upscales, relights, removes objects, and more — each with its own flags and its own

reference page. Read the `--help` output end to end and you will know **what every lever does** and still not know **which lever to pull first**.

This book does the opposite of a reference. It picks **one image** — a promotional poster for a fictional weekend event — and follows it from an empty project on a Tuesday evening to a finished, upscaled, print-ready file, reaching for each feature exactly when a real designer would, and no sooner. By the last page you will have watched the poster grow from a single lantern in the fog to a two-figure scene with a hand-drawn look, and you will know where every tool sits in the arc of making one picture.

If you learn best by watching something get built, start here. You can read this book beside the reference (it names the relevant command whenever it uses one in passing) or entirely on its own.

The poster we are making

Our example is deliberately modest — big enough to exercise a scene, a small cast, and a distinct look; small enough to finish inside one volume.

TERM NIGHT MARKET

A poster for a fictional weekend event. A lantern-lit cobbled lane at night: a food vendor works a wooden cart while a single customer leans across the counter to order. String lights overhead, cool fog beyond, a warm amber glow, and the words NIGHT MARKET across the top. A small, human scene — one hero, one bystander, one prop, and a mood.

We chose a scene with **people, a prop, and a look** on purpose, because that is what exercises the most of plakats: a composition to keep coherent, two figures who must each stand where they are told, an object the model will try to melt into a person, and a treatment that has to survive from draft to print. A single still-life would teach you a third of the program. If you only ever make single-subject images, the middle chapters will still show you the safety rails — you just won't need to climb as high.

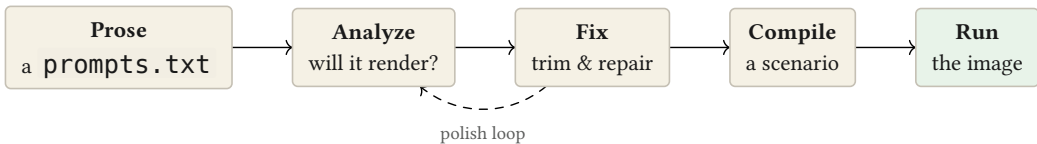
The one idea to take away

If you remember nothing else from this book, remember this: in *plakat* you write in **prose**, and everything else serves the prose or the pixels.

TERM **Prose**

A plain-language description of your scene in a `prompts.txt` file — sentences, not settings. *plakat*'s `compile` command turns prose into a runnable **scenario**; the `analyze` and `fix` commands check and repair the prose before you spend a single render on it. You edit the prose. You almost never hand-edit the machine output.

Most of the frustration people feel with image models comes from editing the wrong thing — poking at a raw prompt string, re-rolling seeds, waiting three minutes to discover the scene was impossible from the start. *plakat*'s answer is a short **authoring loop** that keeps you in prose and tells you what is wrong before it costs you anything.



*The authoring loop. You stay in prose; analyze and fix tighten it **before** a render burns time.*

The shape of the journey

● ● ● *From blank project to finished print*

```
I   Setting Up ..... install · your first image · the prose
project
II  Authoring in Prose ..... write the scene · will it render? · fix it
III Compiling & Running .... compile a scenario · pick a model · tune
IV  Composition & Control .. add figures · a cart & a relation · style
V   Editing & Finishing .... edit · naturalize · upscale · export
VI  Reference ..... directives · model aliases · the polish
loop
```

Each part is a stage every image passes through. You will not use every plakat feature to make **your** poster — few images need all of them — but by the end of this one you will have seen where each fits, and be able to reach for the right one at the right time.

HOW TO READ THE SCREENS

plakat is a terminal program, so the book teaches with faithful monospace **screen** mockups like the one above, rather than screenshots. What you type is shown after a \$ prompt; what plakat prints back is shown plainly. Commands are real and current as of plakat 6.30.0 — type them and they run.

What you need

A computer that can run a diffusion model. plakat is pure Rust and runs on Apple Silicon (Metal), NVIDIA (CUDA), or CPU — slowly, but it **will** finish on CPU. The opening chapter checks your machine and downloads the one small model our first image needs. You do not need an API key to begin: the default prompt enhancer is a small model that runs on your own machine, and every image is generated locally. If you later opt to route the optional prompt-reasoning steps through a hosted model, that text leaves your machine for the provider you name — Chapter 6 is explicit about when that happens and how to keep it fully local (including a larger local model under Ollama).

WHAT YOU LEARNED

- This book follows **one poster**, “NIGHT MARKET”, from `plakat init` to a finished print — learning by watching one image get made, feature by feature in the order you would actually reach for them.
- The central idea: you author in **prose**; `compile` turns prose into a runnable scenario, and the `analyze/fix` loop repairs the prose **before** a render costs you time.
- The journey runs in six parts — setup, authoring, compiling & running, composition & control, editing & finishing, and a short reference — each a stage every image passes through.
- You need only a machine that runs a diffusion model and no API key to start; the book climbs the “control ladder” only as far as the poster needs.

A POSTER, START TO FINISH



PART I

Setting Up

CHAPTER 1

1

Installing plakat, and Your First Image

Every project starts the same way: get the tool onto your machine, confirm the machine can run it, and make **one** image so you know the whole chain works before you invest in a scene. This chapter does exactly that. By the end you will have plakat installed, a clean bill of health from `doctor`, and the first frame of our poster — an empty lantern-lit lane — sitting in an `out/` folder.

Getting plakat

plakat ships three ways. Pick whichever suits how you like to install software; they all land you at the same `plakat` command.

From crates.io (the Rust way)

If you have a Rust toolchain, one command builds and installs the current release. It compiles from source, so budget a few minutes the first time. One thing to get right here: `cargo install` builds a **CPU-only** binary unless you ask for a GPU backend — so add `--features metal` on Apple Silicon (or `--features cuda` on NVIDIA). See the callout below for why this is a build choice, not a runtime one.

● ● ● *Install from crates.io — with a GPU backend*

```
$ cargo install plakat --locked --features metal    # Apple Silicon
# NVIDIA: cargo install plakat --locked --features cuda
# plain `cargo install plakat` installs too — but renders on the CPU
(slow)
  Updating crates.io index
  Compiling plakat v6.30.0
  Finished release [optimized] target(s)
  Installed /Users/you/.cargo/bin/plakat
```

A prebuilt binary (no toolchain)

Every release also publishes prebuilt archives on GitHub — one per platform, each with the right GPU backend already baked in (Metal on the Apple-Silicon archive, CUDA on the `...-cuda` one). Grab the one for your machine, unpack it, and put the binary on your `PATH` — nothing else to select.

● ● ● *Download a release binary*

```
# macOS, Apple Silicon
$ curl -LO https://github.com/vulogov/plakat/releases/download/\
  v6.30.0/plakat-v6.30.0-aarch64-apple-darwin.tar.gz
$ tar xzf plakav6.30.0-aarch64-apple-darwin.tar.gz
$ sudo mv plakav6.30.0-aarch64-apple-darwin /usr/local/bin/
```

The release carries five builds: Apple Silicon (`aarch64-apple-darwin`), Linux on ARM and x86 (`aarch64/x86_64-unknown-linux-gnu`), a CUDA Linux build (`...-cuda`), and Windows (`x86_64-pc-windows-msvc.zip`). A `SHA256SUMS` file sits beside them if you want to verify the download.

From source

To hack on `plakat`, or to pick your own feature set, clone and build. The default build includes the interactive UI, the photo manager, and the fractal engine — but **no GPU backend**, so add one for real speed: `--features metal` on Apple Silicon, `--features cuda` (or `cuda`) on NVIDIA. Trim the defaults with `--no-default-features` for a leaner CLI binary.

● ● ● *Build from source — with the Metal backend*

```
$ git clone https://github.com/vulogov/plakat
$ cd plakat
$ cargo build --release --features metal # Apple Silicon; --features
cuda on NVIDIA
$ ./target/release/plakat --version
plakat 6.30.0
```

Leave the feature off and you get a working binary that renders on the **CPU** — correct, but minutes per image where the GPU is seconds. `plakat doctor` will report `built cpu` so you can catch it at a glance.

THE BACKEND IS A BUILD CHOICE, NOT RUNTIME MAGIC

`plakat` does **not** conjure a GPU backend from bare hardware. The backend is chosen at **build time**: a prebuilt release binary has the right one baked in for its platform, but `cargo install plakat` and a plain `cargo build --release` are **CPU-only** — no GPU backend is on by default. Add `--features metal` (Apple Silicon) or `--features cuda/cudnn` (NVIDIA) to get one. What `plakat` **does** auto-detect is which of the **compiled-in** backends to use at runtime; `--device` overrides that per command. Skip the feature on a Metal Mac and you get a slow CPU binary — and `doctor`'s “build vs runtime” line (below) is exactly what tells you so.

Checking the machine: `doctor`

Before downloading a multi-gigabyte model, ask plakats whether this machine is set up correctly. `doctor` is fully offline by default — it inspects your hardware, your build-versus-runtime device alignment, cache disk usage, and whether the optional tools it can use (ffmpeg, API tokens) are present. It never prints a token’s value, only whether one is set.

● ● ● *plakat doctor*

```
$ plakats doctor
INFO Using Metal device

text-to-image hardware
  backend    metal · features metal
  memory     24 GB unified · budget ~18 GB
  status     healthy — SD 1.5 / SDXL / SD3.5 (low-mem) run here

build vs runtime
  built      metal
  runtime    metal · aligned

cache
  HF cache   ~/.cache/huggingface · 0 B (empty)

tokens
  HF_TOKEN   not set (only gated models need it)
  CIVITAI_TOKEN not set
```

The **build vs runtime** block is the one to read first on a machine you built yourself. `built metal · runtime metal · aligned` means the Metal backend is compiled in and in use. If instead you see `built cpu` on an Apple-Silicon Mac, you installed without `--features metal` — the binary works but renders on the CPU. Rebuild with the feature and this line flips to `metal · aligned`. It is the definitive answer to “why is my Mac rendering so slowly?”

Two follow-up questions are worth asking on a new machine. **Which models can this hardware actually run?** and **how fast will a render be?** plakats answers both without downloading anything.

● ● ● *Will it run, and how fast?*

```
$ plakat doctor --capability
text-to-image hardware
  backend  metal · budget ~18 GB
  sd15      runs      · 5122 native
  sdxl      runs      · 10242 native
  sd35-medium runs      · needs HF_TOKEN
  sd35-large tight     · needs ≥32 GB; use sd35-medium
  flux-dev  won't fit  · → flux-dev-gguf Q4 + --quantize-t5

$ plakat doctor --benchmark
conv2d / matmul / resize at SD-typical shapes ...
extrapolated SD 1.5 wall-time ~9 s / image (28 steps)
```

TERM **Capability report**

`plakat doctor --capability` probes your RAM and backend, derives each model's weight size, and judges **runs** / **tight** / **won't-fit** — naming the one lever that helps when a model is tight. It is the fastest way to know what you can make on this machine before you commit to a download. Add `--json` for a scriptable version.

Choosing the first model

Our first image needs a model. `plakat` recognises short **aliases** — `sd15`, `sdxl`, `flux-schnell`, and many more — each mapping to a full HuggingFace repo. The safest starting model is Stable Diffusion 1.5: it is small, open (no token), fast, and runs on almost anything. We will graduate to SDXL for the finished poster later; for a first proof of life, `sd15` is perfect.

● ● ● *See every model alias plakat knows*

```
$ plakat models aliases
— SD 1.5 —
  sd15, sd-1.5          → stable-diffusion-v1-5/... (base)
— SDXL —
  sdxl                  → stabilityai/stable-diffusion-xl-base-1.0
  sdxl-turbo            → adversarial-distilled, 1–4 steps, no CFG
  pony                  → Pony Diffusion v6 XL (SDXL fine-tune)
— Flux —
  flux-schnell          → 4-step, Apache-2.0
  flux-dev              → gated; larger, higher quality
```

You do not download a model by hand — plakat pulls the weight files it needs on first use and caches them under your HuggingFace cache. If you would rather fetch them ahead of time (say, before going offline), `plakat models pull` does it, and `plakat models ls` shows what is already cached.

The first image

Now the moment of truth. We ask for the seed of our poster — the empty lane, no people yet — with a single, plain sentence. Do not overthink this prompt; we will rebuild it properly, in prose, over the next chapters. Right now we only want to see the machine produce a picture.

● ● ● *Generate the empty lane*

```
$ plakat generate "a lantern-lit cobbled market lane at night, \
  soft fog, warm amber glow, no people" \
  --model sd15 --size 512x512 --seed 1000
INFO Using Metal device
INFO downloading stable-diffusion-v1-5 ... 4 files, 1.9 GB
INFO 28 steps ████████████████████ 28/28 8.7s
✓ wrote ./out/plakat-1000.png
```



Image 01 — the first render: the empty lantern-lit lane. Rough and generic on a one-line prompt, but unmistakably the right scene.

The first run pays a one-time download for the model; later runs skip straight to generation. When it finishes, open `./out/plakat-1000.png`. You should see a narrow cobbled lane, a lantern or two, fog softening the far end — rough, a little generic, but unmistakably the right scene. That roughness is expected: this is a one-line prompt on the smallest model. The whole rest of the book is about turning this proof of life into a poster.

A word on what just happened, because it shapes everything after. `plakat generate` is the **scratchpad**: one sentence in, one throwaway image out, nothing kept. It is perfect for a first pulse-check like this one, and you will keep it nearby for quick scouting. But it is **not** how we will make the poster. From the next chapter on, the images we keep come out of a small, repeatable loop — write prose,

compile it, run the scenario — and **generate** steps aside to be what it is best at: a place to try something fast before committing it to the source.

What plakat wrote beside the image

Look in **out/** and you will find more than a PNG.

● ● ● *What a generation leaves behind*

```
$ ls out/
plakat-1000.png      the image
plakat-1000.json     the recipe: model, seed, steps, sampler, size
```

Every image carries its own recipe. The **.json** sidecar — and a matching block of metadata written **inside** the PNG — records exactly how the image was made, so you can reproduce it, tweak one setting, or hand it to someone else. Two commands live on this metadata: **plakat metadata out/plakat-1000.png** reads it back, **plakat** and **clone out/plakat-1000.png** prints the exact command to regenerate it. We will lean on both in the finishing chapters.

SEEDS MAKE IT REPEATABLE

The **--seed 1000** is why your image will match the recipe next time. Leave the seed off and plakat picks a random one (and records it, so you can recover it). Fix the seed whenever you want to change **one** thing and see only that change — which, in image work, is most of the time.

A note on speed and patience

On Apple Silicon or a decent NVIDIA card, an SD 1.5 image is a handful of seconds. On CPU it can be minutes. plakat will finish either way, but if you are on CPU, prefer **sd15** at **512x512** for everything in the early chapters, keep **--count 1** while you experiment, and save the bigger models for when the composition is settled. The single most expensive mistake in image work is rendering the wrong thing slowly — which is precisely the mistake the next four chapters are built to prevent.

WHAT YOU LEARNED

- Install plakat three ways — `cargo install plakat`, a prebuilt release binary, or from source. The GPU backend is a **build** choice: release binaries bake in the right one per platform, while `cargo install/from-source` are **CPU-only** unless you add `--features metal` (Apple Silicon) or `--features cuda` (NVIDIA). At runtime plakat auto-selects among the backends compiled in; `doctor` shows `built ...` so you can confirm you got the fast one.
- `plakat doctor` health-checks the machine offline; `--capability` says which models **run / are tight / won't fit** here, and `--benchmark` estimates render time — all before any download.
- Models are named by short **aliases** (`sd15`, `sdxl`, `flux-schnell`, ...) and pulled on first use; `sd15` is the safe, fast, token-free starting model.
- `plakat generate "<sentence>" --model sd15 --seed 1000` makes the first image and writes a `.png` plus a `.json` recipe you can later read (`metadata`) or reproduce (`clone`).
- Fix a seed when you want to change one thing at a time — and never render a big version of something you haven't proven small first.

CHAPTER 2

2

The Prose Project

You made an image in the last chapter by typing a sentence straight into `plakat generate`. That is fine for a one-off. But a poster is not a one-off: you will render it dozens of times as you tune the scene, add figures, and try looks, and you want every one of those renders to share the same settings, the same negative terms, the same reproducible seed. The moment you want **the same image, changed a little**, you want a project.

This chapter sets up that project and introduces the two files at the heart of how `plakat` wants you to work — one you write, one the machine writes — and the command that turns the first into the second.

Bootstrapping with `init`

`plakat init` scaffolds a starter project in the current directory: a ready-to-run scenario, a `wildcards/` folder for reusable phrase lists, and a focused `.gitignore` so your renders and model cache don't end up in version control.

● ● ● *Start a project*

```
$ mkdir night-market && cd night-market
$ plakat init
✓ wrote scenario.hjson
✓ wrote wildcards/ (adjective.txt, place.txt, ...)
✓ wrote .gitignore
next: plakat scenario scenario.hjson
```

Open `scenario.hjson` and you will see a heavily commented file: a `model`, a `size`, a `seed`, an `out` directory, a sampler, and one or more **tasks**, each with a prompt. This is a **scenario** — the machine-readable batch format `plakat` actually renders from.

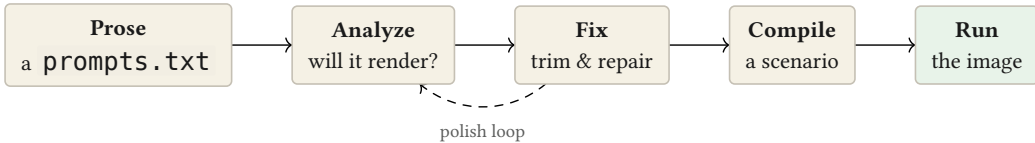
TERM Scenario

An HJSON file describing one or more render **tasks** and the global settings they share (model, size, seed, sampler, output folder). `plakat scenario file.hjson` runs every task in it. It is precise, complete, and verbose — and it is **not** where you want to do your thinking.

You **could** write scenarios by hand, and for a one-task tweak you sometimes will. But a scenario is a poor place to compose. It mixes what you mean (“a foggy lane with a food cart”) with how the machine should render it (steps, guidance, negative terms, budget). Change the scene and you must hand-edit prompt strings, negatives, and per-task settings in lockstep. That is editing the wrong thing.

The file you actually write: `prompts.txt`

plakat's answer is to let you author in prose and **compile** it. You write a plain `prompts.txt` — sentences and a few simple directives — and `plakat compile` turns it into a scenario, enhancing each prompt for the model family, adding sensible negative terms, and fitting everything to the model's token budget.



*The authoring loop. You stay in prose; analyze and fix tighten it **before** a render burns time.*

Here is the beginning of our poster's prose file. The first block, before the first blank line, is the **global block**: settings shared by every scene. Everything after is a **scene** — for now, just the empty lane.

● ● ● *prompts.txt — the global block and one scene*

```
# NIGHT MARKET — poster prose. Edit THIS file; compile the rest.
model: sdxl
size: 1024x1024
seed: 1000
out: ./out

name: empty-lane
A narrow cobbled market lane at night, wet stones catching the light.
Two iron lanterns on stone posts throw a warm amber glow; soft fog
swallows the far end of the lane. Quiet, empty, atmospheric.
```

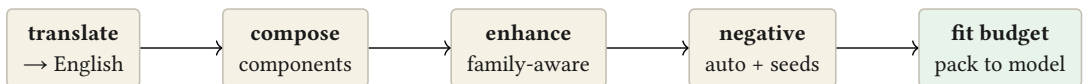
Notice what is **not** here: no comma-separated keyword soup, no “masterpiece, best quality, 8k, highly detailed” boilerplate, no manually tuned negative list. You write what the scene **is**, in sentences. The directives — `model`, `size`, `seed`, `out`, and a scene's `name` — are the few settings prose can't express in words.

TERM Global block vs. scene block

The block before the first blank line sets project-wide defaults (`model`, `size`, `seed`, `out`, `loras`, and reusable pieces). Each later block is one scene: free-text prose plus optional per-scene directives like `name:`, `seed:`, or `negative:`. A scene can override most globals for itself.

What compile does — the short version

Run `compile` and plakat reads the prose, resolves the global settings, and rewrites each scene into a scenario task. We will spend all of Chapter 6 on this; here is just the shape of it, so the next three chapters make sense.



Each prose block becomes one scenario task by passing through these stages — `--explain` shows them.

Each scene passes through the same stages: any non-English is translated, components are composed into one description, the prompt is **enhanced** in a way appropriate to the model family (SDXL likes keyword clusters; Flux and SD3 like prose), a negative prompt is assembled from your seeds plus a curated quality set, and the whole thing is **fit to the model's token budget**. Out comes a scenario.

● ● ● *A first look at compile*

```

$ plakat compile prompts.txt --out scenario.hjson
  compiling 1 scene(s) via local ...
  ✓ 1/1 · empty-lane
  ◆ scene 'empty-lane' · SDXL · sdxl
    translate: (english) · compose: prose only
    enhance: local · negative: auto + 0 seeds
    fit: ~41 tokens (budget ~150, SDXL)
  ✓ compiled → scenario.hjson
$ plakat scenario scenario.hjson
  ✓ ./out/empty-lane-1000.png
  
```

Two commands: `compile` turned prose into a scenario, and `scenario` rendered it. From here on you will almost always edit `prompts.txt` and re-compile — never hand-edit `scenario.hjson`. The scenario is **output**, like a compiled binary. The prose is your source.

YOU CAN STILL GO DIRECT

Nothing stops you from running `plakat generate` for a quick experiment, or hand-writing a scenario for a one-off batch. The prose project is the workflow that **scales** — with a poster you will render many times, it pays for itself by the second afternoon. Use the quick path to explore; move to prose to build.

Why a whole workflow for one picture?

It is fair to ask why an image needs source files, a compiler, and an output format at all. The answer is the thing this book is really about: **the cost of a wrong render**. A diffusion model is slow and a little unpredictable. Every time you change a prompt and hit render, you are placing a bet — a minute or three of compute — that the change will help. Most of the frustration in image work is losing those bets: the scene was too crowded, a word meant nothing to the model, a figure came out standing when you wanted it seated, and you find out **after** the render.

The prose project exists so you can make those bets cheaply and read the odds before you place them. Because your scene is prose, `plakat` can **analyze** it and tell you what will probably fail. Because analysis is cheap, you can fix the prose and re-check in seconds. Only when the prose looks sound do you compile and spend a real render. That loop — write, analyze, fix, compile, run — is the next three chapters, and it is the difference between an evening of waiting and an evening of making.

THE QUICK PATH

`plakat generate "..."` — one sentence, one image. Perfect for exploring an idea or checking a model. Nothing to maintain, nothing reproducible beyond the saved recipe.

THE PROSE PROJECT

`prompts.txt` → `compile` → `scenario` → `run`. Edit prose, re-compile, render many consistent versions. Analyzable, fixable, reproducible. The way to **build** a poster rather than stumble onto one.

WHAT YOU LEARNED

- `plakat init` scaffolds a project: a `scenario.hjson`, a `wildcards/` folder, and a `.gitignore`.
- A **scenario** is the machine-readable batch format `plakat` renders from — precise but a poor place to compose. Treat it as compiled output, not source.
- You author in a plain `prompts.txt`: a **global block** of shared settings, then one **scene block** of prose per image. Write what the scene **is**, in sentences.
- `plakat compile prompts.txt` turns prose into a scenario — translating, composing, enhancing per model family, adding negatives, and fitting the token budget — and `plakat scenario` renders it.
- The whole workflow exists to make the **wrong** render cheap to avoid: prose can be analyzed and fixed before you spend compute, which is what the next three chapters do.

A POSTER, START TO FINISH



PART II

Authoring in Prose

CHAPTER 3

3

Writing the Scene in Prose

A blank lane is a backdrop, not a poster. In this chapter we write the scene properly: we give it a subject, describe the light and the mood, name the pieces that will recur, lean on a word or two we care about, and tell the model what we do **not** want. Everything here is still plain prose in `prompts.txt` — we are composing, not configuring — but by the end the file holds a real market scene ready for the analyze/fix pass in the next two chapters.

Sentences, not keyword soup

If you have used other image tools you may reach for the old habit: market lane, night, lanterns, fog, cobblestones, cinematic, masterpiece, 8k, highly detailed, trending on artstation. Resist it. plakat's compiler enhances your prose for whichever model family you target — it knows SDXL likes some keyword clustering and that Flux and SD3 want flowing description — so your job is to say clearly **what is in the scene and what it feels like**, in sentences.

● ● ● *The scene as prose*

name: market-lane

A narrow cobbled market lane at night, wet stones catching lantern light.

A food vendor works a small wooden cart under a canopy strung with warm string lights; steam rises from a pot. Soft blue fog gathers beyond the cart, and two iron lanterns on stone posts light the foreground. Warm amber glow against the cold night, quiet and inviting.



Image 02 — the scene this prose will compile and render to: a vendor at his cart, warm light against cold fog. You are writing the source; the poster comes later, out of the scenario (Chapter 7).

That reads like a sentence a person would say. It names a subject (the vendor and cart), places the light (lanterns, string lights), sets the palette (warm amber against cold blue), and gives a mood (quiet, inviting). That is enough for the compiler to work with — and, crucially, it is enough for the **analyzer** in the next chapter to reason about.

Naming the pieces: components

Our poster will exist in several versions — the empty lane, the single vendor, the vendor-and-customer, a series with a recurring face. The lane and the cart recur in all of them, and they should look **identical** every time. Retyping their descriptions

invites drift: the cart gains a canopy in one scene and loses it in the next. The cure is to define each recurring piece once, as a **component**, and refer to it by name.

TERM **Component**

A named, reusable fragment of scene description, declared `component.<name>: <text>`. Define it once and refer to it by name from a **directive** — `composition:`, `foreground:`, `objects:`, `background:`, or `relate:`. The compiler substitutes the component’s text wherever it is referenced, so a recurring cart or lane has a single source of truth. Components are one shared namespace whether declared in the global block or inside a scene.

Defining components

You declare components as `key: value` lines, most often in the global block so every scene can reach them. The name after the dot is how you will refer to it.

● ● ● *Defining the recurring pieces*

```
model: sdxl
size: 1024x1024
seed: 1000
out: ./out

component.lane:  a narrow cobbled market lane at night, wet stones
catching
                  lantern light, soft blue fog beyond, two iron lanterns
on
                  stone posts
component.cart:  a small wooden food cart under a canopy strung with
warm
                  string lights, a little chalkboard menu, steam rising
component.vendor: a food vendor in a dark apron, sleeves rolled up
```

A COMPONENT IS NOT SUMMONED BY MENTION

This is the mistake to avoid. Writing the word “cart” in a free-text sentence does **not** pull in `component.cart` — the compiler only substitutes a component where a **directive** references it by name. A component you define but never reference from a directive is simply unused. Referring to components is a deliberate act, and the rest of this section is the four ways to do it.

Composing a scene from components: ``composition:``

The most direct way to build a scene **out of** components is the `composition:` directive. List the components you want, comma-separated, and the compiler joins their descriptions in order — **then** appends whatever free-text prose you add. This is the “compose, then prose” model: the components lay down the reliable, reusable substance; your prose adds the staging and mood on top.

● ● ● *Compose from components, then add prose*

```
name: market-lane
composition: lane, cart, vendor
The vendor works the cart, (warm amber glow:1.3) against the cold night,
quiet and inviting.
```

That scene’s prompt is built from the **full** descriptions of `lane`, `cart`, and `vendor` — every detail you wrote once — followed by the staging sentence. Change `component.cart` in the global block and this scene, and every other scene that lists `cart`, updates at once. A scene can even be pure composition, with no prose at all, when the components already say everything.

BARE NAME OR ``COMPONENT.`` PREFIX

Inside a directive you may write either the bare name (`composition: lane, cart`) or the full form (`composition: component.lane, component.cart`) — the compiler treats them the same, stripping the `component.` prefix. Use whichever reads more clearly to you; this book uses the short bare names.

Components as figures and objects

The other references are the **tiered** directives that Part IV builds on — they, too, take component names. `foreground:` lists the scene’s hero figures, `objects:` the props that must stay separate, `background:` the supporting elements. Each is a comma-list of components, and each does something specific with them (fixing the figure count, reserving a prop its own region) that the composition chapters cover.

● ● ● *The same components, used as figures and objects*

```

name: market-lane
foreground: vendor      # the hero figure (Chapter 9)
objects: cart           # a prop kept separate from the figure (Chapter
10)
composition: lane       # the setting, composed in
A quiet night; the vendor works the cart, warm amber glow.

```

The point for now is that **one** `component.vendor` definition serves every role — hero figure here, a relationship endpoint in Chapter 10, a persona’s description in Chapter

11. Define the vendor once; use him many ways.

Held props fold into their figure

There is one clever behaviour worth knowing early, because it prevents a classic failure. When a component is **attached** to a figure by a holding or wearing relationship — `relate: vendor holding cup` — the compiler folds the held object’s description **into the figure’s own** prompt rather than letting it float as a separate thing. A held paper cup becomes part of “the vendor,” which is exactly where a held object belongs; it stops the model from painting a disembodied cup hovering nearby. (Placement relationships like `behind` or `beside` stay as layout, not folding — the difference is the subject of Chapter 10.)

● ● ● *A held object travels with the figure*

```

component.cup: a steaming paper cup
name: market-lane
foreground: vendor
relate: vendor holding cup    # 'cup' folds INTO the vendor's
description
The vendor offers the cup across the cart.

```

Local overrides

Components live in one namespace, but a scene may declare its own — with the same name — to override the global one just for that scene. Global definitions load first, so a scene-local `component.vendor` shadows the shared one. This is how a single poster in a series can dress the vendor differently (a raincoat for the “wet night” edition) without disturbing the others.

● ● ● *Overriding a component for one scene*

```
# global: component.vendor: a food vendor in a dark apron
name: rainy-night
component.vendor: a food vendor in a yellow oilskin raincoat, hood up
foreground: vendor
composition: lane
Rain streaks the lantern light; the vendor hunches over the cart.
```

Taken together, components are how a project stays coherent as it grows: one definition per recurring piece, referenced by name from wherever it is needed, overridable where a scene genuinely differs. When the vendor gains a customer in Chapter 9, the cart becomes a separated object in Chapter 10, and the vendor becomes a persona for the series in Chapter 11, each of those is the **same** component, reused — never retyped, and never drifting.

Leaning on a word: weighting

Sometimes one element matters more than the rest and the model underweights it — the amber glow washes out, or the fog takes over the frame. *plakat* understands **attention weighting** right in the prose: wrap a phrase in parentheses with a number, and the model pays it proportionally more (above 1.0) or less (below) attention.

● ● ● *Attention weights in prose*

```
name: market-lane
The lane opens before us. A food vendor works the cart,
(warm amber glow:1.3) against the (cold blue fog:0.9),
quiet and inviting.
```

(warm amber glow:1.3) tells the model to push the amber; (cold blue fog:0.9) eases the fog back so it frames rather than floods. Use this sparingly — a scene full of competing weights fights itself. One or two deliberate emphases is plenty, and the compiler preserves your exact weights through enhancement and budget-fitting so a (phrase:1.3) you wrote survives to the render untouched.

WEIGHTS ARE NOT VOLUME KNOBS

A weight nudges attention, it does not guarantee an outcome. (amber glow:1.8) will not force a sunset; it just makes the model lean harder on the words you already wrote. If a concept refuses to appear at 1.3, the fix is usually clearer **description**, not a bigger number.

Splitting attention with BREAK

When two subjects bleed into each other — the vendor's colours smearing onto the cart, say — you can tell the compiler to encode them as separate chunks with the keyword **BREAK**. Everything before **BREAK** is encoded independently of everything after, which keeps distinct elements from contaminating one another.

● ● ● *Separating elements with BREAK*

A food vendor in a dark apron works the cart.
BREAK
 A wooden food cart under a warm string-lit canopy, chalkboard menu.

BREAK is a blunt instrument — reach for it only when elements are genuinely bleeding together. For most scenes, clear prose and a component or two do the job. The heavier machinery for keeping figures apart — regions, and control-generate — waits in Part IV, where the poster actually needs it.

Saying what you don't want: negatives

Every render also has a **negative** prompt: things to steer away from. plakat builds a sensible one for you automatically — a curated quality set that suppresses the usual failure modes (extra limbs, watermarks, blur) — so you rarely start from nothing. When your scene has a specific thing to avoid, add it with a **negative:** directive and it joins the automatic set.

● ● ● *A scene-specific negative*

name: market-lane

The lane opens before us. A food vendor works the cart, warm amber glow against the cold night, quiet and inviting.

negative: daylight, blue sky, modern signage, cars, crowd

Our poster is a **night** market, so **daylight** and **blue sky** are worth excluding explicitly; we want it intimate, so **crowd** and **cars** go too. Keep negatives short and specific to **this** scene — the automatic quality terms already handle the generic defects, and a bloated negative list dilutes the ones that matter.

DON'T NEGATE IN THE POSITIVE

Write what you **want** in the prose, and what you **don't** in the negative. Phrases like “a lane with no people” in the positive prompt often backfire — the model reads “people” and paints some. Put **people**, **crowd** in the negative: instead. The analyzer in the next chapter flags this exact mistake.

Where the scene stands

Our `prompts.txt` now has real content: shared components for the lane and cart, a scene that places a vendor in warm light against cold fog, one or two deliberate weights, and a tight negative list. It reads like a description a person would give a painter. That readability is not just for you — it is what lets plakat **check** the scene before rendering it, which is exactly where we go next.

WHAT YOU LEARNED

- Write scenes as **sentences** describing what is present and how it feels — the compiler handles model-appropriate phrasing, so keyword-soup and “masterpiece, 8k” boilerplate are unnecessary.
- Define recurring pieces once as `component.<name>:`, then **reference them from a directive** — `composition:` (compose-then-prose), `foreground:`, `objects:`, `relate:`. A bare mention in prose does NOT pull a component in; a scene-local definition overrides the global one.
- Lean on a phrase with attention weighting, `(phrase:1.3)` up or `:0.9` down, and separate bleeding elements with `BREAK` — both sparingly; the compiler preserves your weights to the render.
- Add scene-specific **negative:** terms on top of plakat’s automatic quality set, and describe what you **want** in the positive — never “no people,” which tends to summon them.
- Readable prose is not just tidy; it is what makes the scene **analyzable** — the subject of the next chapter.

CHAPTER 4

4

Will It Even Render?

Here is the scene that plays out for every image-maker sooner or later. You write an ambitious prompt. You hit render. You wait — one minute, two, three — watching the progress bar crawl. And what comes back is a mess: a figure fused into a table, a cart with a person's arm growing out of it, six things fighting for the centre of a frame that only had room for two. You burned the compute, and the scene was doomed from the first word. You just didn't know it yet.

plakat has a command whose entire job is to tell you **before** you spend that render. It reads your prose, reasons about what will probably fail, grades the scene, and hands you specific fixes — and it renders nothing.

The feasibility report: `compile --analyze`

Let us deliberately over-reach, the way one does at the end of a good idea, and ask the poster to hold everything at once.

● ● ● *An over-stuffed scene*

```
name: market-lane
A narrow cobbled market lane at night. A food vendor in an apron works a
wooden cart while a customer leans in to order and two children run past
and a cat sits on a barrel and a musician plays in the background,
string
lights and lanterns and a chalkboard menu and steam and reflections in
the
wet stones, a bustling crowd beyond, no empty space, everything
happening
at once.
```




Image 03 — the over-stuffed scene the analyzer warns about: seven subjects fighting for one frame, figures fusing, detail dissolving into noise.

Now, instead of compiling and rendering it, ask whether it **can** work.

● ● ● *plakat compile --analyze*

```
$ plakat compile prompts.txt --analyze
scene 'market-lane' — feasibility 3/10 · HIGH RISK
```

- ▶ OVER-STUFFED (high): 7+ distinct subjects (vendor, customer, two children, cat, musician, crowd) compete for one frame. SDXL reliably renders 2–3 focal subjects; the rest will blur or fuse.
fix: cut to the vendor + one customer; imply the rest as "a few distant figures in the fog."
- ▶ PERSON+CROWD FUSION (med): "bustling crowd beyond" behind a busy foreground tends to grow extra limbs on the focal figures.
fix: push the crowd fully into the background and out of focus.
- ▶ NEGATION-IN-POSITIVE (low): "no empty space" describes an absence; the model can't paint "no." fix: describe density positively, e.g. "a full, lively lane," or move it to negative:.

recommendation: reduce to two figures + the cart; re-run --analyze until the grade is LOW RISK before you compile.

That report cost seconds and zero renders. It caught the three things that would have wrecked the image: too many subjects, a crowd that fuses with the foreground, and a “no” the model cannot paint. Each risk comes with a **concrete** fix, not a scold.

TERM Feasibility grade

`compile --analyze` scores a scene from 1–10 and labels it LOW / MODERATE / HIGH RISK, then lists the specific failure modes it predicts — over-stuffing, figure fusion, rare or invented object names, negation-in-positive, hard poses, count ambiguity, token bloat — each with a targeted fix. It writes nothing and renders nothing. It is a critic, run before the cost.

Why this is worth a whole command

The value of `--analyze` is not that it is clever; it is that it is **cheap and early**. Consider the arithmetic. A HIGH-RISK scene on SDXL might take two minutes to render and come back unusable, and you might try it three or four times — re-rolling the seed, nudging a weight — before you accept that the **scene**, not the seed,

was the problem. That is ten minutes of waiting to learn what `--analyze` tells you in five seconds.

THE WALL YOU DON'T WANT TO HIT

The worst outcome in image work is not a bad image — it is a **slow** bad image, followed by the realisation that the prompt was impossible for this model from the start. `--analyze` exists so you meet that wall as a sentence on your screen, not as three wasted renders and a sinking feeling.

The polish loop

`--analyze` is not a one-shot verdict; it is the companion to a loop. You read the risks, edit the prose to answer them, and re-analyze — as many times as it takes to bring the grade down. Only when the scene reads LOW RISK do you compile and spend a real render.

Let us walk the loop on our poster. We take the analyzer's advice and cut the scene to its heart: the vendor, one customer, the cart, the light.

● ● ● *The scene, trimmed — and re-analyzed*

```
name: market-lane
```

```
A narrow cobbled market lane at night. A food vendor works a wooden cart under warm string lights; a single customer leans in to order. Two iron lanterns light the foreground; soft blue fog and a few distant figures beyond. Warm amber glow, quiet and inviting.
```

```
$ plakat compile prompts.txt --analyze
  scene 'market-lane' — feasibility 8/10 · LOW RISK
```

```
  ▶ note (low): two figures + a cart is well within SDXL's reliable range.
```

```
    For the tightest control of who stands where, consider control-generate
```

```
      (Ch. 9) — optional; the scene will render without it.
```

```
recommendation: good to compile.
```

From 3/10 to 8/10 by **removing**, not adding. This is the most common lesson the analyzer teaches: over-stuffing is the default failure, and the fix is almost always subtraction. The children, the cat, the musician, the crowd were all lovely ideas; none of them survived contact with a frame that could hold two people and a cart. The poster is stronger for their absence.

Where analyze fits the arc

Think of `--analyze` as the read-through you do before committing to a draft. It does not write your scene and it does not render it; it tells you, honestly and early, whether the scene you wrote is one this model can make. Run it whenever a scene grows — after you add a figure, after you pile on detail, before you reach for a bigger, slower model. It is the cheapest insurance in the whole toolchain.

But reading a report and hand-editing prose to answer it is still work, and some of the fixes are mechanical — rename a rare word, delete a negation, trim a clause. The next chapter shows how `plakat` can apply those safe fixes **for** you, back up your originals, and — this is the part that matters for a long project — record **why** each change was made, so the reasoning of your polish loop is never lost.

WHAT YOU LEARNED

- `compile --analyze` reads your prose and predicts what will fail — over-stuffing, figure fusion, rare names, negation-in-positive, hard poses — grading the scene 1–10 and labelling it LOW / MODERATE / HIGH RISK, all without rendering.
- Its value is being **cheap and early**: it turns “three wasted renders and a sinking feeling” into a five-second sentence you can act on.
- Use it as a **loop** — read the risks, edit the prose, re-analyze — and compile only once the scene reads LOW RISK.
- The most common cure is **subtraction**: over-stuffing is the default failure, and our poster went from 3/10 to 8/10 by cutting a crowd down to two figures and a cart.
- Analyze is the read-through before the draft; the next chapter automates its safe fixes and records the reasoning behind them.

CHAPTER 5

5

Fixing the Prose — and Remembering Why

The analyzer tells you what is wrong. Some of what it finds is judgment — **which** subject to cut, how to reframe a crowd — and stays your call. But some of it is mechanical: a rare word the model won't know, a negation in the positive prompt, a clause of pure token bloat. *plakat* can apply those safe fixes for you, without touching a thing it isn't sure about, and — for a poster you will polish over many sessions — it can **remember why each change was made**. That memory is where a small, quiet tool called *smysl* earns its place.

Applying the safe fixes: `--analyze --fix`

Add `--fix` to an analyze run and plakat proposes safe, high-confidence text edits, finds each offending phrase in the exact source file it lives in, backs that file up first, applies the edit in place, and reports what changed where. Anything it can't fix safely — splitting an over-stuffed scene, choosing which figure is the hero — it leaves for you, clearly labelled.

Suppose our lane picked up a couple of these mechanical problems: a fanciful name for the cart and a bloated boilerplate tail.

● ● ● *The prose with fixable problems*

```
name: market-lane
A narrow cobbled lane at night. A food vendor works a gastro-nocturne
provisions barrow under warm string lights; a customer leans in to
order.
Soft fog beyond. Warm amber glow, masterpiece, best quality, ultra
detailed, 8k, trending on artstation.
negative: no daylight
```

● ● ● *compile --analyze --fix*

```
$ plakat compile prompts.txt --analyze --fix
scene 'market-lane' — feasibility 5/10 · MODERATE RISK
  ▶ RARE NAME (med): "gastro-nocturne provisions barrow" is invented;
the
  model will hallucinate. fix → "wooden food cart"
  ▶ TOKEN BLOAT (low): "masterpiece, best quality, ultra detailed, 8k,
trending on artstation" adds tokens, dilutes the scene. fix → cut
  ▶ NEGATION-IN-POSITIVE (low): "no daylight" belongs in negative.

--fix applying safe auto-fixes to the source prose...
✓ prompts.txt (backup → prompts.txt.1)
  rare/invented name → "gastro-nocturne provisions barrow"
    → "wooden food cart"
  token bloat → "masterpiece, best quality, ultra detailed, 8k,
trending on artstation" → (removed)
  → re-run --analyze to confirm the grade improved.
```

Three things happened, and each matters:

It backed up your original

Before editing, plakat copied `prompts.txt` to `prompts.txt.1` (and `.2`, `.3` on later runs). Your words are never destroyed; you can always diff against the backup or restore it.

It edited the right file

If your prose is split across files with `@include` (Chapter 6), `--fix` traces each phrase to the **actual** file it lives in and edits that one — not a copy, not the wrong include. The offending clause is found verbatim and replaced verbatim.

It respected your language

plakat’s prose is often multilingual — a Russian scene description with English negative terms is common. `--fix` repairs a phrase **in its own language**: a Russian phrase is fixed in Russian, an English one in English. It never quietly translates your words while “fixing” them.

WHAT `--fix` WILL NOT TOUCH

`--fix` only applies edits it is confident are safe: renaming a rare word, cutting boilerplate, moving a negation. It never restructures your scene, never invents content, never edits a component name or a directive. Splitting an over-stuffed poster, or deciding the vendor is the hero and the customer the bystander, remains yours — it is reported under “needs your hand,” not silently changed.

Remembering why: the smysl corpus

Here is a problem that only shows up on a project you return to. You polish the prose over three evenings. On the fourth you look at the file and cannot remember why the cart is a “wooden food cart” and not the vivid name you first wrote — or whether you already dealt with that crowd, or why a clause you liked is gone. The edits survive; the **reasoning** evaporated into a terminal you closed days ago.

plakat can keep that reasoning. When `--fix` runs, it records each decision as a small provenance note in a sidecar file — a **smysl corpus** — beside your prose.

TERM smysl corpus

A sidecar file (<name>.smysl) that records the **why** behind your polish loop: a `smysl` document is a light AI→AI→Human knowledge format. In `plakat` it captures each applied fix as a claim **grounded in** the risk it addressed, plus the budget decisions made at compile time. It is a memory of the polish loop — not a file you author. You keep writing prose; the corpus records the reasoning underneath it.

SMYSL IS A SIDECAR, NOT A NEW SYNTAX

You never write `smysl` by hand, and it never replaces your prose or your scenario. It is a record that accumulates **underneath** the authoring loop, so the polish history is queryable later. If you never look at it, nothing changes about how you work. This is the only role `smysl` plays in `plakat` — the memory of prompt polishing.

The corpus is **born** at the fix step, because that is where decisions accumulate, and it **grows** across runs rather than being overwritten — polish the prose again next week and the new decisions are merged in beside the old, with identical decisions de-duplicated automatically.

● ● ● *The corpus written beside the prose*

```
$ plakat compile prompts.txt --analyze --fix
...
✓ prompts.txt (backup → prompts.txt.1)
smysl corpus → prompts.smysl (2 fix-finding record(s), 0 open)

$ cat prompts.smysl
@finding f/risk-3eh4u { status: speculative }
~ rare/invented object name — model will hallucinate

@claim c/fix-9a2kd { status: derived, grounds: [f/risk-3eh4u] }
~ replace "gastro-nocturne provisions barrow" with "wooden food cart"
```

Read that middle line the way `plakat` intends it: the **fix** (“use wooden food cart”) exists **because of** the **finding** (“rare name — the model will hallucinate”). The edit

is grounded in the risk it answered. That grounding is the whole point — three weeks later you can ask **why** a phrase is the way it is and get an answer, not a shrug.

Asking the corpus: `compile --trace`

Because the reasoning is recorded, you can query it. `compile --trace "<phrase>"` answers a question every returning author asks: **why is this in — or gone from — my prompt?** It runs no model and renders nothing; it reads the corpus and the scene and explains.

● ● ● *Why did this change?*

```
$ plakat compile prompts.txt --trace "wooden food cart"
2 unit(s) mention "wooden food cart":

• c/cart [claim, Speculative]
  a small wooden food cart under a warm string-lit canopy
• c/fix-9a2kd [claim, Derived]
  replace "gastro-nocturne provisions barrow" with "wooden food
cart"
  ← grounded in f/risk-3eh4u
    (rare/invented object name — model will hallucinate)
```

The trace shows both where the phrase lives in your scene **and** the decision that put it there. When we reach budget-fitting in Chapter 8, the same `--trace` will also explain why a phrase was **dropped** to fit a model's token limit — the corpus records the budget decisions too. That is its whole ambition: to make the polish loop's history answerable, so a poster you return to after a month still makes sense.

Where the loop has taken us

We have written a scene in prose, analyzed it before spending a render, and fixed its safe problems while keeping a record of why. The prose now reads clean and LOW RISK. That is the moment to leave authoring and **compile** — to turn this proven prose into a scenario and finally make the image. That is Part III.

WHAT YOU LEARNED

- `compile --analyze --fix` applies only the **safe**, mechanical fixes — renaming a rare word, cutting boilerplate, moving a negation — backing up your original to `prompts.txt.N` first and reporting exactly what changed where.
- It edits the right source file (tracing phrases through `@include s`) and repairs each phrase **in its own language**; structural changes stay yours, reported under “needs your hand.”
- The **smyzl corpus** (`<name>.smyzl`) records the **why**: each fix as a claim grounded in the risk it answered. It is a sidecar memory of the polish loop, not a syntax you author — the only role smyzl plays in `plakat`.
- The corpus is born at `--fix` and **accumulates** across runs (identical decisions de-duplicated), so a long polish history is preserved rather than overwritten.
- `compile --trace "<phrase>"` answers “why is this in — or gone from — my prompt?” with no render, reading the corpus to explain both scene content and decisions.



PART III

Compiling & Running

CHAPTER 6

6

Compiling to a Scenario

Our prose is clean and the analyzer is happy. Now we turn it into something plakat can render: a scenario. This chapter is the compiler in full — how it enhances each scene for the model, how to see exactly what it will do before it does it, how to split a growing prose file across includes, and how to keep the whole thing honest with a validation pass. By the end, `prompts.txt` becomes a `scenario.hjson` you can run with confidence.

The basic compile

At its simplest, `compile` takes a prose file and writes a scenario.

● ● ● *Compile the poster*

```
$ plakat compile prompts.txt --out scenario.hjson
  compiling 1 scene(s) via local ...
  ✓ 1/1 · market-lane
  ◆ scene 'market-lane' · SDXL · sdxl
    translate: (english) · compose: 2 components
    enhance: local · negative: auto + 5 seeds
    fit: ~96 tokens (budget ~150, SDXL)
  ✓ compiled → scenario.hjson
```

The trace to the right of each `◆` is worth reading: it names the model family it resolved (SDXL), how many components it folded in, which enhancer it used, how many negative seeds you supplied on top of the automatic set, and — the line we will care about in Chapter 8 — how the final prompt fit the model’s token budget.

If you leave off `--out`, `compile` writes `<stem>.hjson` beside the prose; pass `-` to print the scenario to your terminal instead.

COMPILE CAN ALSO IMPROVE THE PROMPT

Compile’s job is prose → scenario, but it can go one step further. `compile --improve` *enhances, then automatically improves*: it renders, scores the result, rewrites the prompt to raise its aesthetic quality, and writes the **winning** prompt into the scenario. The `smysl` corpus is its memory, so it never re-tries a change it already rejected. Chapter 8 covers the improve loop in full; here, just know that the same `compile` you run to produce a scenario can also polish it.

The enhancer, and choosing a provider

The **enhance** stage is where a scene’s plain prose becomes a prompt tuned to the model family. That stage runs through a language model — the “provider.” `plakat`

defaults to a small model that runs on your own machine (no key, a one-time download); you can run a larger local model under Ollama, or point it at a hosted model if you have a key.

● ● ● *Picking the enhancement provider*

```
# on-device, no key, nothing leaves the machine (default)
$ plakat compile prompts.txt --compile-provider local

# a larger LOCAL model via Ollama – still nothing leaves the machine
$ plakat compile prompts.txt --compile-provider ollama:qwen2.5-coder:14b

# a HOSTED model, if you have a key in the environment
# (your prose is sent to that provider – see the note below)
$ plakat compile prompts.txt --compile-provider deepseek
$ plakat compile prompts.txt --compile-provider gemini

# let plakat choose what's configured
$ plakat compile prompts.txt --compile-provider auto
```

WHAT LEAVES YOUR MACHINE — AND WHAT DOESN'T

Image generation is always local. Every render runs on your own GPU or CPU and never touches a provider. The only thing that can leave your machine is the **text** of the optional prose-reasoning steps — enhancement here, and later `--analyze`, `--improve`, and the layered planner — and only if you point one at a **hosted** provider. Be clear-eyed about what that means:

- If you choose a hosted provider (`deepseek`, `gemini`, or `auto` when a key is configured), the request — your prose and the prompt being enhanced or judged — **is sent over the network to that provider**. With `local` or `ollama:<model>`, nothing leaves the machine.
- Hosted models are usually **metered**: those calls can **cost money**. The account, the API key, the token usage, and the bill are yours to set up, watch, and pay — plakat neither meters nor caps them for you.
- Any such request is governed by **that provider's** terms of service and privacy policy, and is subject to their handling and retention — not plakat's. plakat neither sees nor controls what a provider does with what you send.
- To keep everything on your machine while still using a capable LLM, run one under **Ollama** and select it with `--compile-provider ollama:<model>`.
- These reasoning arcs are **entirely optional**. plakat compiles and renders without any of them — `--no-enhance` `--no-negative` (below) skips the model altogether, and `--analyze` / `--improve` only run when you ask. Whether to use an external provider, and for what, is your call and your responsibility.

DETERMINISTIC COMPILES

If you want a compile with **no** language model at all — byte-for-byte reproducible, offline, instant — pass `--no-enhance` `--no-negative`. The prose is assembled verbatim, components folded in, weights preserved, and the budget still fitted (that part is model-free). It is the mode continuous-integration checks use, and a fine default when your prose is already exactly what you want.

Seeing what it will do: `--explain`

Before spending even the small cost of an enhancement pass, you can ask compile to **show its work**: the model family it resolved for each scene, the exact system prompt it will send the enhancer, and the deterministic negative it will build. It calls no model and writes nothing.

● ● ● *compile --explain*

```
$ plakat compile prompts.txt --explain
— scene "market-lane" · family SDXL
[positive system]
You write prompts for Stable Diffusion XL (dual CLIP, ~150-token range).
Mix natural language with keyword clusters. Aim 60–150 tokens. ...

[negative (deterministic)]
lowres, bad anatomy, extra limbs, watermark, text, blurry, ...
↳ negative seeds: daylight, blue sky, modern signage, cars, crowd
```

`--explain` is how you answer “why did my prompt come out like that?” without guessing. If a scene is being enhanced in a way you don’t want, this is where you see the instruction responsible — and often the fix is a clearer sentence in the prose, or a different `model` so a different family profile applies.

A dry run and a validity check

Two more no-render inspections earn their keep on a big project. `--dry-run` summarises what a compile **would** do — how many scenes, how many model calls, a rough token estimate — so you can gauge cost before you spend it. And `--check` validates that a scenario (compiled or hand-written) actually loads and uses known task types, a fast offline gate for continuous integration.

● ● ● *Estimate, then validate*

```
$ plakat compile prompts.txt --dry-run
compile dry-run · prompts.txt · provider local
  - market-lane · family SDXL · 2 LLM call(s) · ~180 tok
total: 1 scene · 2 call(s) · ~180 tokens (rough)

$ plakat compile scenario.hjson --check
✓ scenario is valid (loads · known task types)
```

Splitting the prose: `@include`

As a poster grows into a series, one flat `prompts.txt` gets unwieldy. `plakat` lets you split prose across files and pull them together with `@include`, optionally passing values in.

● ● ● *Composing prose from parts*

```
# prompts.txt
@include shared/globals.txt
@include shared/components.txt

name: market-lane
The lane opens before us. A vendor works the cart, warm amber glow.
```

Includes are why `--fix` goes to the trouble of tracing a phrase back to the file it truly lives in: when your cart's description sits in `shared/components.txt`, a fix to the cart edits **that** file, not a flattened copy. Your prose stays modular and your history stays honest.

The scenario it produced

Finally, look at what `compile` wrote. You will rarely edit this by hand, but you should recognise it.

● ● ● *A peek inside scenario.hjson*

```
{
  model: sdxl
  size: 1024x1024
  seed: 1000
  out: ./out
  scheduler: euler-a
  tasks: [
    {
      name: market-lane
      prompt: "night market lane, wet cobblestones, a food vendor at a
        wooden cart under warm string lights, (warm amber glow:1.3),
        soft blue fog, two iron lanterns, cinematic night lighting"
      negative: "daylight, blue sky, cars, crowd, lowres, extra limbs,
        ..."
    }
  ]
}
```

Everything in there came from your prose: the globals from the global block, the prompt from your sentences (enhanced, with your `(warm amber glow:1.3)` weight intact), and the negative from your seeds plus the automatic quality set. This is compiled output. When you want to change the image, you change the prose and recompile — and the two commands that make that a habit are `--watch`, which recompiles whenever you save the prose, and `--diff`, which shows what a recompile **would** change against an existing scenario.

● ● ● *A tight edit loop*

```
$ plakat compile prompts.txt --watch
👁️ watching prompts.txt — Ctrl-C to stop
✓ compiled → prompts.hjson           # every time you save
```

WHAT YOU LEARNED

- `plakat compile prompts.txt --out scenario.hjson` turns prose into a scenario; the per-scene trace names the family, components, enhancer, negatives, and budget fit.
- The **enhance** stage runs through a provider — `local` (on-device, default), `deepseek/gemini` (hosted), or `auto`; `--no-enhance` `--no-negative` gives a deterministic, offline, reproducible compile.
- Inspect before you spend: `--explain` shows the family and exact system prompt, `--dry-run` estimates cost, and `--check` validates that a scenario loads.
- `@include` splits prose across files (with value passing); this is why `--fix` and `--trace` can trace a phrase to the file it truly lives in.
- The `scenario.hjson` is **compiled output** — recognise it, but edit the prose and recompile; `--watch` rebuilds on save and `--diff` previews a recompile's changes.
- The same `compile` can also **improve**: `--improve` enhances, then renders, scores, and rewrites the prompt to raise its aesthetic quality, writing the winner into the scenario (Chapter 8).

CHAPTER 7

7

Running It, and Choosing a Model

A scenario is ready; now we render it, and we make the first big craft decision of the project: which model. plakat speaks to a whole family of diffusion models, and they are not interchangeable — they differ in size, in speed, in how they like to be prompted, and in how much text they can hold. This chapter runs the poster and then chooses, deliberately, the model it will wear to print.

Running the scenario

One command renders every task in a scenario.

● ● ● *Render the poster from the scenario*

```
$ plakat scenario scenario.hjson
task 1/1 · market-lane
  INFO 30 steps ██████████ 30/30 22.4s
✓ ./out/market-lane-1000.png
```



The market-lane poster — the output of `plakat scenario`, not a one-off `generate`. This is the same image you saw take shape from prose in Chapter 3: prose → compile → scenario produced it, and re-running the scenario reproduces it exactly.

That is the whole point of the workflow this book has been building. The image is not something you coaxed out of a prompt box and hope to find again — it is the **deterministic output** of a source file you own. The prose compiled to this

scenario; the scenario rendered this poster; and both steps are repeatable. If you only want to see what **would** run — how many tasks, at what size, which model — add `--dry-run`. Because each task carries a name and a seed, re-running the same scenario reproduces the same images, while bumping the global `seed` produces a fresh, reproducible set.

THE SCENARIO IS WHERE IMAGES COME FROM

From here on, every poster in this book is a **scenario render**. You will still meet `plakat generate` as a quick scratchpad for scouting a seed or a style, but the images you keep — the ones you tune, compose, and print — come from `plakat` out of `scenario`, driven by the prose you compiled. `Generate` explores; the scenario produces.

The model families

`plakat` groups models into families, and the family — more than any single model — determines how you prompt and what you can expect.

sd15	Small, fast, token-free. 512 ² native. The learning and CPU-friendly default; a hard 77-token text limit.
sd21	Like SD 1.5 but 768 ² native, v-prediction.
sdxl	1024 ² native, dual text encoders, ~150-token effective range. The reliable general-purpose choice — our poster's model.
sdxl-turbo	Adversarial-distilled SDXL: 1–4 steps, no guidance. Very fast drafts.
pony	An SDXL fine-tune (a full SDXL, budget and all) tuned for stylised character art.
sd35-medium / large	- MMDiT with a T5-XXL encoder — a much larger ~256-token budget and strong prose comprehension. Gated (needs HF_TOKEN).
flux-schnell / dev	- Transformer models, no CLIP token cap — write flowing prose. <code>schnell</code> is 4-step and Apache-licensed; <code>dev</code> is larger and gated.

sana / pixartEfficient large-context prose transformers (Gemma-2 / T5-XXL).
Big budgets, light on memory.**TERM Model family**

A group of models that share a text encoder and prompting style: SD 1.5 (77-token CLIP, keyword-friendly), SDXL (dual CLIP, 150 tokens), SD3 (T5-XXL, 256 tokens), Flux (transformer, no CLIP cap), Cascade, and the efficient prose transformers Sana and PixArt. The family sets how you prompt and how much text survives to the render.

The token budget follows the model

That “how much text survives” is not a footnote — it is the reason your careful prose sometimes gets trimmed. Each family has an effective token budget, and plakart fits your prompt to the **actual model you chose**. SD 1.5 holds about 77 tokens; SDXL about 150; the T5-driven families 256 or more. Ask a rich scene of SD 1.5 and the tail of your prompt is at risk; ask the same of SDXL and it fits with room to spare.

● ● ● *Same prompt, different budgets*

```
$ plakart compile prompts.txt --model sd15 --dry-run
    market-lane · family SD15 · budget ~77 tokens
$ plakart compile prompts.txt --model sdxl --dry-run
    market-lane · family SDXL · budget ~150 tokens
$ plakart compile prompts.txt --model sana --dry-run
    market-lane · family SD3 · budget ~256 tokens
```

PROPER BUDGET FOR THE PROPER MODEL

plakart keys the budget to the real model, aliases and all — **pony** gets SDXL’s 150-token budget (it **is** an SDXL fine-tune), and **pixart** and **sana** get the T5-scale 256, not the small default. You do not manage this; you only benefit from it. The next chapter shows what happens at the budget’s edge — and how plakart trims to fit without silently dropping your subject.

Sizes and the native-resolution rule

Each family was trained at a native resolution, and straying far from it invites artefacts — duplicated heads, stretched bodies, repeated horizons. The rule of thumb is simple: render near the model's native size, then **upscale** (Chapter 14) to reach print dimensions.

sd15 / sd21	512 ² / 768 ² . Small and forgiving.
sdxl / pony	1024 ² . Our poster renders here, then upscales.
sd35 / flux / sana	1024 ² native; larger with care and memory.

DON'T FIGHT THE NATIVE SIZE

Rendering SD 1.5 at 1024² or SDXL at 512² is the fastest way to a two-headed figure. Set `--size` to the family's native square (or a matched aspect via `--aspect / --base`), and leave the big final dimensions to the upscaler. `plakat` warns when a control-generate draft size and model are mismatched, for exactly this reason.

What will run here?

Before committing to a heavier model, ask the machine. `doctor --capability` (from Chapter 1) judges each model **runs** / **tight** / **won't-fit** on your hardware and names the lever that helps when something is tight. For the gated models — SD 3.5, Flux dev — you will also need a HuggingFace token in your environment; `plakat` tells you when one is missing rather than failing deep in a download.

● ● ● *Match ambition to hardware*

```
$ plakat doctor --capability
  sdxl          runs      · 10242 ← our choice
  sd35-medium   runs      · needs HF_TOKEN
  sd35-large     tight    · use PLAKAT_SD3_LOWMEM=1, or sd35-medium
  flux-dev      won't fit · → flux-dev-gguf Q4 + --quantize-t5
```


RUNNING BIG MODELS ON MODEST MACHINES

A tight model is often still reachable. SD 3.5 has a low-memory mode (PLAKAT_SD3_LOWMEM=1) that fits it on a 24 GB Metal machine; Flux has quantised GGUF variants. `doctor --capability` names the specific lever each time — you rarely have to guess.

The poster's model

For NIGHT MARKET we choose **SDXL**. It reads our two-figure, one-prop night scene comfortably, renders at a print-friendly 1024², prompts in the mix of prose and keyword clusters our compiler already targets, and — importantly for the control work coming in Part IV — has the strongest ecosystem of pose and style adapters. It is the reliable middle of the road, and a poster wants reliability more than it wants the last five per cent of a harder-to-drive model.

We set it once, in the global block, and every scene inherits it:

● ● ● *Committing to SDXL*

```
model: sdxl
size: 1024x1024
scheduler: euler-a
```

With the model chosen and the scene rendering, only one thing stands between us and a good frame: the dozen small dials — seed, steps, guidance, sampler — that turn a correct image into the **right** one. That is tuning, and it is the next chapter.

WHAT YOU LEARNED

- `plakat scenario scenario.hjson` renders every task; names and seeds make runs reproducible, and `--dry-run` previews without rendering.
- Model **families** — SD 1.5, SDXL, SD3, Flux, Cascade, Sana/PixArt — set how you prompt and how much text survives; the family matters more than the individual model.
- The token budget follows the **actual** model (SD 1.5 77, SDXL 150, T5 families 256), and `plakat` keys it correctly even for aliases like `pony` (SDXL) and `sana / pixart` (T5-scale).
- Render near a model's **native resolution** (SDXL at 1024²) and upscale later; rendering far off-native invites duplicated or stretched anatomy.
- `doctor --capability` says what runs / is tight / won't-fit here and names the lever (low-mem mode, quantised GGUF) that helps; gated models need `HF_TOKEN`.
- We commit the poster to **SDXL**: reliable, 1024² native, well-supported by the pose and style adapters Part IV will use.

CHAPTER 8

8

Tuning the Image

The scene is correct; now we make it **good**. Tuning is the set of small dials that turn a plausible render into the one you want to print — the seed that frames it well, the sampler that sharpens it, the step count that earns its time, the guidance that balances fidelity against life. None of these change **what** is in the image; they change how well it is realised. And nearly all of them live where the rest of your poster lives: in the **scenario**. You tune by editing the scenario (or the prose it came from) and re-running it — staying inside the prose → compile → scenario loop, not stepping outside it.

Where the dials live: the scenario

The four dials you reach for most — the seed, the step count, the guidance, and the sampler — are all scenario directives. Set them in the global block and every task inherits them; change one and re-run `plakat scenario` to see only that change.

seed	The initial noise the image grows from. A different seed is a different composition of the same scene. Fix it to change one thing at a time.
steps	Denoising steps. More is smoother, to a point; 28–35 is plenty for SDXL. Past ~40 you mostly buy time.
guidance	How hard the model obeys the prompt (CFG). ~7 is a good middle; lower is looser and more natural, higher is more literal and can scorch.
scheduler	The sampler: <code>default · ddim · euler-a · unipc</code> . <code>euler-a</code> often sharpens SD/SDXL at the same step count; <code>unipc</code> (DPM++) converges fast.

● ● ● *Tuning dials in the scenario — edit, then re-run*

```
model: sdxl
seed: 1000
steps: 30
guidance: 6.5
scheduler: euler-a

$ plakat scenario scenario.hjson      # re-render with the new dials
✓ ./out/market-lane-1000.png
```

Because the seed is fixed, changing `guidance` from 7.5 to 6.5 and re-running shows you **exactly** what the guidance did — same composition, softer obedience — with nothing else moving. That is the discipline the scenario gives you: reproducible, one-variable tuning. Edit the source, re-run, compare. No hunting for a frame you can't get back.

Seeing several framings at once

To compare compositions rather than settings, ask the scenario for a **set**: bump `count`, and each task renders several images at stepped, reproducible seeds. Pick the framing you like and pin its seed back in the global block.

● ● ● *A reproducible set of seeds, from the scenario*

```
count: 4          # renders seeds 1000, 1001, 1002, 1003 – all
reproducible

$ plakat scenario scenario.hjson
✓ ./out/market-lane-1000.png ... market-lane-1003.png
```

The scratchpad: scouting with `generate`

There is a faster, looser way to **explore** — and it is worth being clear about what it is. `plakat generate` renders a single prompt straight from the command line. It is a **scratchpad**: it sits outside the prose → compile → scenario loop, it keeps nothing you have to maintain, and the images it makes are throwaways. Its job is to let you try a seed, a variation, or a sampler in seconds, so you can carry the **setting you liked** back into the prose or the scenario — where the poster you keep actually comes from.

GENERATE EXPLORES; THE SCENARIO PRODUCES

Use `generate` the way a painter uses a corner of the canvas to test a colour. Scout freely — but the finished poster is always a **scenario** render, driven by your compiled prose. Anything you discover on the scratchpad (a good seed, a step count, a sampler) you fold back into the scenario; you do not ship the scratch.

A few things the scratchpad is good at:

Nudging a frame you like: subseed

When a seed is **almost** right, a **subseed** blends a second noise into it for “the same image, nudged” — a slightly different drape or pose without losing the framing.

● ● ● *Scout a variation, then keep the seed*

```
$ plakat generate "<the market-lane prompt>" --model sdxl \
  --seed 1000 --subseed 77 --subseed-strength 0.15
```

`--subseed-strength` in `[0,1]` sets the blend; `0.05–0.2` nudges, `1` is the subseed's own noise (SD 1.5 / SDXL). Found the nudge you want? The seed and subseed are recorded in the image's sidecar — carry them into the scenario and the batch reproduces it.

Contact sheets and culling

To eyeball many at once, `--count` with `--grid` writes a contact sheet; to let the machine triage, `--keep-best` scores them by an aesthetic predictor and keeps the top few. Both are scouting conveniences.

● ● ● *Scout wide, keep the best few*

```
$ plakat generate "<prompt>" --model sdxl --count 12 --keep-best 3 --
grid
$ plakat rank out/ --top 5          # or score a folder you already have
```

Going fast while scouting

When you want frames **now**, the `--fast` presets bundle a distillation LoRA with the right sampler and step count so a draft lands in a handful of steps instead of thirty — ideal for the scratchpad, then drop back to full steps in the scenario for the frame you commit to.

● ● ● *Four-to-eight-step scouting drafts*

```
$ plakat generate "<prompt>" --model sdxl --fast lightning-sdxl-8
$ plakat generate "<prompt>" --model sdxl --fast hyper-sdxl-4
```

(There are Flux presets too — `hyper-8`, `turbo-alpha` — and `lcm-sd15` for SD 1.5.)

Letting plakat find the better prompt: ``compile --improve``

Culling a batch tells you which render scored best. The natural next question is whether plakat can **use** that score to make the prompt better — and it can, without ever leaving the loop. `compile --improve` runs the whole pipeline in one command: it **enhances** your prose into a prompt (the translation pass), then **improves** it — rendering, scoring with the same aesthetic predictor `--keep-best` uses, proposing one small rewrite, keeping it only if the score truly rises — and finally **writes the winning prompt back into the scenario**. The improvement lands in the HJSON you render, not just on the screen.

● ● ● Enhance, improve, and emit — one command

```
$ plakat compile prompts.txt --improve --improve-model sdxl
— scene market-lane
  · aesthetic 6.18 → "wet stones" → "shimmering puddles" → 6.39 kept
  · aesthetic 6.27 (reverted)
✓ compiled (improved) → prompts.hjson
```

The corpus is what keeps it from marching

Left to itself, an automatic rewrite loop **wanders**: it tries a change, undoes it, tries it again — a *death march* that spends renders and converges on nothing. plakat's cure is the smysl corpus from Chapter 5, used now as an **active memory** rather than a record you read later. Every move the loop tries — kept or *reverted* — is written to the corpus, and before proposing the next one the loop consults that record and **refuses** to re-try a rejected change or its reverse. The corpus is created if it does not exist and consulted on every run, so a second `--improve` never re-walks ground the first one already ruled out. The loop always stops with a **reason** — a plateau, the pass budget spent, or no fresh move left — and reports the best prompt it found.

TERM The corpus as tabu memory

The same `<name>.smysl` sidecar that records **why** a fix was applied now also records every rewrite the improve loop tried and how it scored. That history is a *tabu list*: the loop cannot repeat a move it already spent, which is precisely what turns a death march into a search that converges. The corpus is not a side-effect of the process — it *is* the process's memory.

Steering the loop

- improve-all · --improve-scene a,b** Improve every scene, or a named subset (the flag repeats and takes commas). Default: the first scene only.
- improve-seeds N** The aesthetic score is noisy, so each candidate is rendered and scored on N seeds (default 2) and averaged — a kept/rejected verdict you can trust.
- improve-skip-good** Consult the corpus and SKIP any scene already at the best score a prior run reached, so recompiling doesn't re-polish what's done. Use `--improve-target N` for an absolute bar instead.
- improve-passes N** How many rewrites to try per scene before stopping (default 6).
- keep-compiled-images** Archive every candidate it renders — normally scored then discarded — so you can see the whole trajectory as image files.

WHAT --IMPROVE TUNES — AND WHAT IT DOESN'T

`--improve` optimizes the **prompt**: wording, composition, palette, light. It renders a plain image of the emitted prompt — without your LoRAs or control-generate — so it tunes the **writing**, not the painted finish. Reach for it to find a stronger prompt; reach for the persona LoRAs and the control ladder (Chapters 11 and 9) for the look.

Tuning that lives in the prose

Not every dial is a setting; one is **prose**, and so it rides inside the loop with everything else. **Prompt scheduling** lets the image start as one thing and finish

as another — lay down a composition early, let detail take over late — written right in the scene’s text, so it survives compile into the scenario prompt.

● ● ● *Scheduling, written in the prose*

```
# in prompts.txt — switch from A to B at 40% of the steps
a [foggy empty lane : bustling lit stall : 0.4] at night

# alternate between two every step
a lane lit by [lanterns | string lights]
```

[a:b:when] swaps **a** for **b** once the render passes the fraction **when**; [a|b] alternates each step (SD 1.5 / SDXL). Because it is prose, it is analyzable, fixable, and reproducible like the rest of your scene — a tuning tool that never leaves the loop.

The dial you don't turn: fitting the budget

One more thing happens every time you **compile** — inside the loop, with no dial to set — and it is the reason your rich prose survives to the render intact. When a prompt runs past the model’s token budget (Chapter 7), plakat **packs** it to fit, in a way you can audit rather than silently dropping the end of your sentence.

TERM Model-free budget packing

When a prompt exceeds the model’s budget, plakat first tries a deterministic pack: it scores each phrase by attention weight, position, and whether it is generic filler, then drops the lowest-value spans — **never the subject** — until the prompt fits. It records exactly what it cut and why. Only if trimming filler isn’t enough does it fall back to asking the language model to reword.

● ● ● *A budget trim you can read — at compile time*

```
$ plakat compile prompts.txt --model sd15
◆ scene 'market-lane' · SD15
  ... prompt was ~86 tokens (over the SD15 ~77 budget) —
    packed model-free to ~75, dropped 2:
      dramatic lighting [low-value]; ultra detailed [low-value]
```

Notice **what** it dropped: generic quality-boosters, not your vendor, cart, or fog. And because the decision is recorded in the smysl corpus from Chapter 5, you can ask about it later — the same `--trace` that explained why a phrase **is** in your prompt now explains why one is **gone**.

● ● ● *Why did that phrase disappear?*

```
$ plakat compile prompts.txt --trace "dramatic lighting"
• o/pack-3eh4u [observation]
  budget pack 'market-lane': dropped 2 spans to fit SD15 in 77
tokens
  used 75/77; dropped: dramatic lighting [low-value] ...
```

The corpus traces both the changes you made and the budget the model imposed — the two forces that shape a final prompt. On SDXL our poster fits with room to spare; on a tighter model, this is what keeps the trim honest. With the scene tuned in the scenario and the budget fit at compile, the loop has done its work — and we can start adding figures.

WHAT YOU LEARNED

- The core dials — **seed**, **steps**, **guidance**, **scheduler** — are scenario directives: edit the scenario (or its prose) and re-run `plakat scenario` for reproducible, one-variable tuning. Use `count` for a reproducible set of framings.
- `plakat generate` is a **scratchpad outside the loop** — scout a seed, a subseed variation, a `--fast` draft, or cull with `--keep-best / rank`, then fold the setting you liked back into the scenario. You scout with `generate`; you ship with `scenario`.
- `compile --improve` closes an **automatic** loop inside `compile`: `enhance` → `improve` → `emit`. It renders, scores with the aesthetic predictor, keeps only rewrites that truly help, and **writes the winner into the scenario**. The `smysl` corpus is its memory — every tried move, kept or reverted, so it never re-marches. `--improve-skip-good` leaves already-good scenes alone.
- Prompt scheduling (`[a:b:when]` , `[a|b]`) is **prose**, so it stays inside the loop — analyzable and reproducible like the rest of the scene.
- Budget packing happens at **compile**, inside the loop: it drops generic filler first, never the subject, records what it cut, and `--trace` explains a dropped phrase just as it explains a kept one.

A POSTER, START TO FINISH



PART IV

Composition & Control

CHAPTER 9

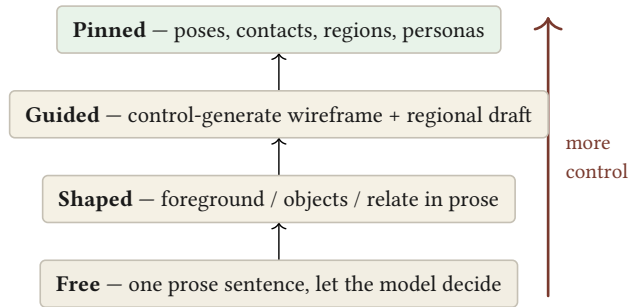
9

More Than One Figure

Up to now our poster has had one focal figure — the vendor — and the model has placed him wherever it liked. That is fine for one person. Add a second, and diffusion models start to struggle: two figures blur into each other, swap limbs, or drift to opposite corners of the frame. Our poster needs the vendor **and** a customer, standing where we want them. This chapter is about taking that control without overreaching for it — climbing the control ladder exactly one rung.

The control ladder

Before adding machinery, it helps to see the whole range of control plakat offers, because the craft is in using **the least that works**.



The control ladder. Start free; climb only as far as the image needs — every rung costs effort.

Every rung up costs effort and can cost naturalness — a heavily pinned scene can look stiff. So we climb only as far as the image needs. One figure, free-form, needed no rungs. Two figures who must not merge need one: **shaping** them in prose, and letting plakat's composition pass keep them apart.

Naming the figures: `foreground:`

The first tool is a directive that tells the compiler how many deliberate people are in the scene and who they are.

TERM foreground:

A list of the scene's intended focal figures, e.g. foreground: vendor, customer. It tells the compiler the exact number of people to plan for — which becomes the render's figure count — so a second person is a **decision**, not an accident the model may or may not honour.

● ● ● *Two named figures*

```

component.vendor:  a food vendor in a dark apron, sleeves rolled up
component.customer: a customer in a wool coat, seen from behind

name: market-lane
foreground: vendor, customer
A vendor works the cart under warm string lights while a customer
leans in to order. Warm amber glow, soft fog beyond.

```

Naming the two figures does two things: it fixes the count (the model will not add a third or lose one), and it hands each figure a stable description you can reuse — the same instinct as the components from Chapter 3, now applied to people.

Turning on composition: `control-generate`

For two figures to reliably hold their positions, plakat can run a **composition pass** — it plans where each figure goes, drafts the layout, and then finishes the image. You switch it on with one directive.

TERM **control-generate**

A directive (`control-generate: sdxl`) that enables plakat's multi-stage composition pipeline: it builds a pose/placement plan for the named figures, draws a regional draft that puts each where it belongs, and hands that draft to a finish pass. The value names the **draft** model. Pair it with `control-generate-size` for that model's native resolution.

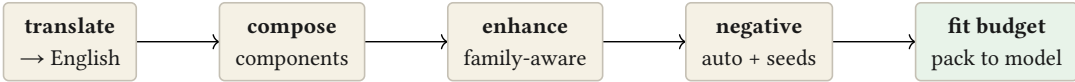
● ● ● *Enabling the composition pass*

```

name: market-lane
foreground: vendor, customer
control-generate: sdxl
control-generate-size: 1024x1024
A vendor works the cart while a customer leans in to order.

```

Under the hood this runs in stages — a placement plan, a regional draft that paints each figure in its own area of the canvas, and a finishing pass that harmonises the whole. You do not manage the stages; you describe the scene and choose the draft model. `--explain` shows the compiled control directives if you want to see them.



Each prose block becomes one scenario task by passing through these stages — `--explain` shows them.

Posing the figures

The plan needs to know **how** each figure stands, and here prose does most of the work: describe the posture and the compiler reads it. “A vendor **standing** at the cart” and “a customer **leaning in**” become distinct poses in the plan. You can also state them explicitly when you want no ambiguity.

● ● ● *Posture from prose, or stated outright*

```

# read from the prose verbs ...
A vendor stands at the cart; a customer leans across the counter.

# ... or pinned explicitly, one per foreground figure
control-generate-figure-poses: standing, leaning
  
```

plakat knows a vocabulary of postures — standing, sitting, kneeling, leaning, reaching, carrying, and more — and builds a pose skeleton for each figure that the draft honours.

AN HONEST WORD ABOUT HARD POSES

Standing figures are where pose control is strongest. Seated, crouched, and other unusual poses are genuinely harder for SDXL’s pose adapters — the draft may fight you. If a pose refuses to hold, two things help: the SD 1.5 draft model (`control-generate: sd15`, at 512x512) has stronger pose control for difficult postures, and simplifying the pose (“leaning in,” not “crouched and twisting”) costs you nothing. Our vendor and customer both stand or lean, which is the reliable range.

The regional toggle

By default the composition pass paints each figure in its own region so they don’t merge. Occasionally — a very simple two-figure scene — you may want to let the finish pass blend more freely. The `control-generate-regional` toggle controls that.

● ● ● *Regional draft on or off*

```
control-generate-regional: true    # each figure in its own area
(default)
control-generate-regional: false   # let the finish pass blend more
```

For our poster we keep it on: the vendor belongs at the cart on the right, the customer at the counter on the left, and we want that separation to survive the render. If figures ever come out **too** separated — like cut-outs pasted side by side — turning regional off and leaning on clear prose can soften the seam.

Where the poster stands



Image 04 — two figures held apart: the vendor at the cart and a customer leaning in, each keeping position instead of merging.

Run it, and the draft now reliably shows two people: the vendor at his cart, the customer leaning in to order, each holding position instead of melting together. We climbed one rung — named the figures, enabled composition, let posture flow from the prose — and stopped there. The scene did not need pinned coordinates or personas yet.

But there is still a lie in the frame: the cart. The model wants to treat the vendor and his cart as one blob of “person-with-stuff,” fusing the wooden cart into his body.

Keeping a **prop** distinct from a **person** is a different problem from keeping two people apart — and it is the next chapter.

WHAT YOU LEARNED

- Climb the **control ladder** only as far as the image needs — every rung costs effort and can cost naturalness; two non-merging figures need just one rung.
- `foreground: a, b` fixes the scene's figure **count** and names each person, so a second figure is a decision the render honours, not an accident.
- `control-generate: <draft-model>` (with `control-generate-size`) turns on the composition pass — plan, regional draft, finish — that keeps figures in their places; you describe the scene, `plakat` runs the stages.
- Posture flows from prose verbs (standing, leaning, reaching...) or can be pinned with `control-generate-figure-poses: ;` **standing/leaning are the reliable range**, and the `sd15` draft is stronger for hard poses.
- `control-generate-regional` keeps each figure in its own area (default on); turn it off only if figures come out looking pasted-on.

CHAPTER 10

10

A Cart, and a Relationship

A diffusion model has a strong instinct to fuse. Give it “a vendor at a cart” and it will happily paint one object — a person-shaped mass with cart-like bits growing out of the torso — because to the model the vendor and his cart are one region of “stuff that goes together.” Our poster needs them **distinct**: a person, and beside him a separate wooden cart he is working. It also needs the two figures to relate — the customer facing the vendor, not staring past him. Both are jobs for a small, specific vocabulary that this chapter covers.

The fusion problem, concretely

You have already met fusion’s cousin. In Chapter 4 the analyzer warned about “person + cargo fusion” — a big carried object melting into the figure holding it. A cart beside a vendor is the same failure in a new costume: the model blends the prop into the person. Prose alone struggles here, because “a vendor at a wooden cart” **reads** as one thing. You have to tell plakat that the cart is its own object.

Declaring a separate object: ``objects:``

The `objects`` directive names props that must stay distinct from the figures — the compiler plans them their own place in the composition rather than letting them collapse into a person.

TERM `objects``:

A list of scene props that must render as **separate** objects, not fused into the figures — e.g. `objects` cart``. The compiler gives each its own region in the composition plan, with a gap from the figures, so a cart stays a cart beside the vendor instead of melting into him. The tool for the thing the model wants to absorb.

● ● ● *The cart as its own object*

```
component.vendor: a food vendor in a dark apron, sleeves rolled up
component.cart:  a wooden food cart with a canopy and a chalkboard menu

name: market-lane
foreground: vendor
objects: cart
control-generate: sdxl
control-generate-size: 1024x1024
A vendor works beside his cart under warm string lights.
```



Image 05 — the cart as its own object: a separate wooden cart beside the vendor, not a person-shaped mass with cart parts growing out of it.

With the cart declared as an object, the composition pass reserves it a region of the canvas next to the vendor — the wooden cart on one side, the person on the other, each rendered as itself. The blob becomes two things.

OBJECTS VS. HELD PROPS

Not every prop needs **objects**: . A small thing a figure **holds** — the paper cup the vendor hands over — travels with the figure and is fine in prose. Reserve **objects**: for the large props the model tries to **absorb**: a cart, a vehicle, a market stall. If a prop keeps fusing into a person, that is the signal to promote it to an object.

Relating the figures: `relate:`

Two figures in a frame are not yet a **scene** — a scene has them interacting. Our customer should face the vendor across the cart, leaning in to order. The `relate:` directive states that relationship so the composition honours it.

TERM `relate:`

A relationship between two named components, written `relate: <a> <relationship> <b>` — e.g. `relate: customer facing vendor`. The compiler uses it to orient and place the figures so they interact as described, rather than standing back-to-back or ignoring each other.

● ● ● *The customer faces the vendor*

```
name: market-lane
foreground: vendor, customer
objects: cart
relate: customer facing vendor
control-generate: sdxl
control-generate-size: 1024x1024
The customer leans across the cart to order; the vendor reaches out
a paper cup. Warm amber glow, soft fog beyond.
```

Now the plan knows the geometry of the moment: vendor at his cart, customer opposite and facing him, the cart between them as its own object. That is the full staging of our poster's hero shot — two people, one prop, one interaction — expressed in three short directives on top of plain prose.

Contacts: where figures touch

When figures don't just relate but **touch** — a hand on a shoulder, an arm around a companion — `control-generate-figure-contacts` pins that contact so the limbs meet where they should. Our customer and vendor don't touch (the cart is between them), so we don't need it here, but it is the tool when an embrace or a handshake must land.

● ● ● *Pinning a physical contact*

```
# e.g. two figures side by side, one arm around the other
control-generate-figure-contacts: vendor arm-around customer
```

An honest word on the ceiling

It would be dishonest to promise that any arrangement of figures, objects, and relations renders perfectly. Some do not. Two **coupled wheeled vehicles** — an engine towing a separate trailer — sit right at the edge of what today’s SDXL composition can hold; the model wants to make them one four-wheeled thing no matter how you declare them. Deeply **seated** figures in a busy scene are similarly hard. When you reach that edge, the honest path is not to fight the render but to change the plan: simplify the arrangement, split an impossible scene into two images, or — for a truly fixed layout — reach for the higher rungs of the ladder (pinned poses, a control pre-image) that a specialised job needs.

READ THE ANALYZER BEFORE YOU CLIMB

`compile --analyze` (Chapter 4) knows these ceilings. It flags fused-vehicle and person-cargo risks **before** you render, and its advice — “use a control pre-image for two coupled vehicles,” “split the over-stuffed scene” — is exactly the guidance above, delivered early. When composition gets ambitious, analyze first.

Our poster is comfortably within the reliable range: two upright figures, one separated prop, a facing relationship. It stages cleanly. What it does **not** have yet is a consistent **look** and, for the series we want to make, a vendor whose face stays the same from poster to poster. That is style and identity — the next chapter.

WHAT YOU LEARNED

- Diffusion models **fuse** a prop into the person beside it; a cart at a vendor is the “person + cargo fusion” the analyzer warned about, wearing a new costume.
- `objects: <prop>` declares a prop that must render **separately** — the compiler reserves it its own region with a gap from the figures, so a cart stays a cart.
- `relate: <a> <relationship> <b>` (e.g. `customer facing vendor`) states how two figures interact, so they orient toward each other instead of ignoring the moment.
- `control-generate-figure-contacts` pins limb-level touches (an arm around a shoulder) when figures physically meet — not needed when a cart sits between them.
- Some arrangements sit at the **ceiling** — coupled wheeled vehicles, deep seated poses in a crowd; the analyzer flags these early, and the honest fix is a simpler plan or a higher rung, not a fight with the render.

CHAPTER 11

11

A Look, and a Face That Stays

Our poster is composed: two figures, a cart, a relationship, warm light. What it lacks is a **look** — a deliberate visual treatment that makes it a poster and not a photograph — and, because we want a **series** of NIGHT MARKET posters, a vendor whose face is the same in every one. This chapter covers both: how to reach for a style, whether borrowed, trained, or transferred; and how to pin a person's identity so it survives from image to image.

Reaching for a style

The quickest way to a look is to ask for one. The `--look` and `--genre` flags send plakat to find a matching style adapter — a small trained file called a **LoRA** — and apply it, so “watercolour” or “ink poster” or “vintage travel print” becomes a treatment rather than a word the base model half-understands.

● ● ● *Ask for a look*

```
$ plakat generate "<the market-lane prompt>" --model sdxl \  
  --look "vintage screen-print poster" --smart-discovery  
↳ discovered style LoRA: retro-screenprint-xl (strength 0.8)  
✓ ./out/plakat-<seed>.png
```

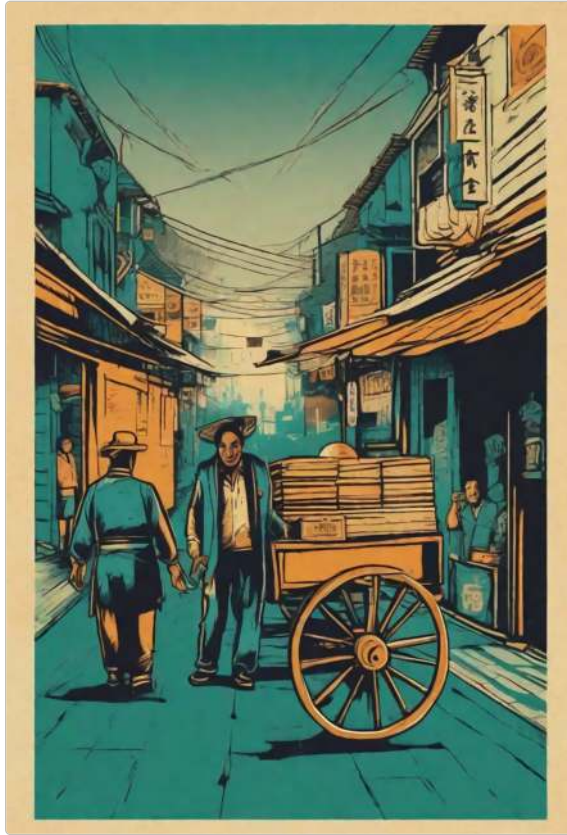


Image 06 — the same scene as a bold two-colour screen-print poster: a look is a treatment applied over the composition, not a new picture.

--smart-discovery is worth adding: a small local judge re-ranks the candidates to pick a genuine **style** adapter and reject character or person LoRAs that would hijack your subject. For reproducible or offline runs, --offline restricts discovery to what is already cached.

TERM LoRA

A small trained adapter that nudges a base model toward a style, subject, or concept, layered on at generation time. plakat can **find** one for you (--look / --genre), or you can name specific LoRAs and stack them. Many LoRAs need a **trigger word** in the prompt to activate.

Naming a LoRA — and its trigger

When you have a specific LoRA in mind — downloaded, or trained yourself — name it directly in the scenario, and give it its **trigger word**, the token it was trained to respond to. Forget the trigger and the LoRA loads but does nothing.

● ● ● *A named LoRA with its trigger*

```
model: sd-xl
loras: ink-poster-xl:0.65
lora-trigger: inkposter

name: market-lane
foreground: vendor, customer
objects: cart
An inkposter night market: a vendor at his cart, a customer leaning in.
```

Two details matter here. The `:0.65` sets the LoRA’s **strength** — how hard it pulls; 0.6–0.8 is a sane range, and pushing to 1.0 often overwhelms the scene. And `lora-trigger:` injects the trigger word (`inkposter`) so the adapter actually fires. `plakat` keeps the trigger verbatim and reserves room for it in the token budget, so your careful prose isn’t crowded out by it.

A TRIGGER WORD IS NOT OPTIONAL

The single most common “my LoRA does nothing” bug is a missing trigger word. If a named LoRA has no visible effect, check its documentation for the trigger and add it with `lora-trigger:`. `plakat` auto-detects some (like LCM speed LoRAs); style and character LoRAs usually need you to name theirs.

Two other roads to a style

Borrowing a LoRA is one road; there are two more, for when you want a look that is truly yours.

Transfer a style from a reference image

If you have an image whose **look** you want — a print you admire, an earlier poster — `stylize` transfers its style onto your content without a trained adapter at all.

● ● ● *Style from a reference*

```
$ plakat stylize --in out/market-lane.png \
  --ref refs/old-travel-poster.jpg --out out/styled.png
```

Train your own style LoRA

And if you have a body of work in a consistent style — say a dozen of your own ink drawings — you can **train** a LoRA on them, so “inkposter” means **your** ink, not a stranger’s. `plakat style train` builds an SDXL or SD 3.5 style LoRA from a folder of examples; the result is a file you name in `loras:` like any other.

● ● ● *Training a style is a real option*

```
$ plakat style train --data ./my-ink-drawings \
  --out ./loras/my-ink-xl --model sdxl
```

(There is a parallel path for a **specific object or character** — textual-inversion embeddings via `plakat embedding`, injected with `generate --embedding` — when a single subject, not a whole style, is what you want to pin.)

A face that stays: personas

Now the harder identity problem. A NIGHT MARKET **series** wants the same vendor in every poster — same face, same build — not a new stranger each render. A seed does not give you that; change anything else and the face drifts. `plakat`’s answer is a **persona**: a reproducible synthetic person you define once and place into scenes.

TERM Persona

A controllable synthetic person, defined once in a `PersonaSpec` and reused across images so the **same** face and build appear every time. `plakat persona new` scaffolds a spec; `multiperson` places one or more personas into a generated scene, each pinned to a location — the way to give a poster **series** a consistent cast.

● ● ● *Define the vendor once, reuse him*

```
$ plakat persona new vendor --out personas/vendor.hjson
✓ scaffolded personas/vendor.hjson (edit the traits, then lint)
$ plakat persona lint personas/vendor.hjson
✓ valid

# place the fixed vendor (and a customer) into the scene
$ plakat multiperson "a night market lane, warm string lights" \
  --at "vendor:right at the cart" \
  --at "customer:left, leaning in"
```

With the vendor a persona, every poster in the series shows the same man at his cart. `multiperson` pins each persona to a relative spot (“right at the cart,” “left, leaning in”) and lets a scene-aware planner place anyone you leave unpinned. If you have a **reference photo** of the face you want, it can anchor the persona’s identity to that face.

IDENTITY VS. A ONE-OFF FACE

For a single poster you don’t need a persona — a good seed and a clear description are enough. Personas earn their keep the moment you want the **same** person **again**: a series, a campaign, a character across a comic (plakat makes those too). Reach for identity control only when continuity is the requirement.

The poster, dressed

NIGHT MARKET now has its look — a screen-print ink treatment, borrowed or trained — and, for the series, a vendor whose face will not wander. Compositionally

and stylistically, the image is **done**: it says what we want, staged how we want, in the treatment we chose. What remains is finishing — editing the last small flaws, making the render feel like a made object rather than a machine output, enlarging it to print size, and signing it. That is the final part.

WHAT YOU LEARNED

- `--look / --genre` (with `--smart-discovery`) find and apply a **style LoRA** so a named treatment becomes real; `--offline` keeps discovery reproducible.
- Name specific LoRAs in `loras: name:strength` (0.6–0.8 is sane) and **always** give a style/character LoRA its `lora-trigger: word` — the top cause of “my LoRA does nothing.”
- Two other roads to a look: `stylize` transfers a style from a **reference image** with no adapter, and `plakat style train` builds a LoRA from **your own** body of work.
- A **persona** pins a synthetic person’s identity across images — `persona new` to define, `multiperson` to place — the way to give a poster **series** a consistent cast; reach for it only when continuity is required.
- With a look and a stable face, the poster is compositionally done; what remains is finishing — the last part of the book.

A POSTER, START TO FINISH



PART V

Editing & Finishing

CHAPTER 12

12

Editing What You Have

Not every problem is worth a re-render. When a poster is ninety per cent right — the composition holds, the light is good — but one thing is wrong (a garbled hand, a distracting sign, a face that came out soft), you do not throw the image away and roll the seed again. You **edit** the pixels you have. plakart has a full set of surgical tools for exactly this, and reaching for them instead of re-rolling is often the difference between finishing tonight and chasing a better seed until midnight.

Changing the whole image a little: `img2img`

The broadest edit takes an existing image and re-renders it **guided by itself**, so the composition survives while the treatment shifts. A low strength nudges; a high strength reinvents.

● ● ● Guided re-render

```
$ plakat img2img out/market-lane.png \
  --prompt "warmer amber light, softer fog" --strength 0.35
```

TERM Strength (img2img)

How far an `img2img` render is allowed to travel from the original, `0.0–1.0`. Low (`0.2–0.4`) preserves the composition and adjusts tone or detail; high (`0.6+`) keeps only the rough layout and repaints the rest. Start low — you can always push.

LOW STRENGTH ON FINISHED ART

A whole-image `img2img` at high strength will happily **undo** the careful composition you built over ten chapters. On a cohesive poster, keep strengths low and prefer the **localised** edits below. The lesson `plakat` learned the hard way: re-rendering a whole cohesive image tends to make it worse, not better.

Fixing one spot: masks and inpaint

Most edits should touch **one region**, not the whole frame. Give `img2img` a `--mask` and it changes only the masked pixels — the rest is untouched. This is **inpainting**, and it is how you fix a bad hand or repaint a single object without disturbing anything else.

You need a mask — a black-and-white stencil of the region. You can paint one, or let `plakat` make one by **clicking** the thing you mean, using Segment-Anything.

● ● ● *Select by clicking, then inpaint*

```
# click the garbled hand → a mask
$ plakat segment out/market-lane.png --point 620,540 \
  --out mask.png
# repaint only that region
$ plakat img2img out/market-lane.png --mask mask.png \
  --prompt "a clean hand holding a paper cup" --strength 0.8
```

Removing something entirely: `remove`

When the fix is **deletion** — a stray sign, a smudge, an object that cluttered the frame — `remove` erases it and fills the hole seamlessly, in one command. Point at it, box it, pick it by depth, or name it in words.

● ● ● *Four ways to say what to remove*

```
$ plakat remove out/market-lane.png --point 210,300 # click it
$ plakat remove out/market-lane.png --box 0,0,180,400 # box it
$ plakat remove out/market-lane.png --what "modern sign" # name it
```

`--what` is open-vocabulary: an object detector finds the thing you named, and the region is inpainted away while the rest of the poster is preserved. It is the fastest way to declutter a frame after the fact.

Swapping the background: `replace-bg`

Sometimes the subject is perfect and the **background** is not. `replace-bg` mattes the subject out, generates (or loads) a new background, and composites the subject back over it — decontaminating the edges so there is no halo.

● ● ● *Keep the vendor, change the night behind him*

```
$ plakat replace-bg out/vendor-cut.png \
  --prompt "a deep foggy market lane at night, bokeh string lights" \
  --keep vendor
```

Faces: `restore-faces` and `faceswap`

Faces are where the eye goes first, so plakat gives them dedicated tools. When a face renders soft or slightly wrong — common on small figures far from the camera — `restore-faces` detects each face, refines it with a diffusion pass, and composites it back, sharper and truer, without touching the rest of the image.

● ● ● *Sharpen the faces only*

```
$ plakat restore-faces out/market-lane.png
↳ SCRFD found 2 faces · refined · composited
```

And when you need a **specific** face — the persona from the last chapter, or a fixed model across a series — `faceswap` transplants a source face onto the figure in your image, aligned and colour-matched so it belongs.

● ● ● *Put a specific face on the vendor*

```
$ plakat faceswap out/market-lane.png --source faces/vendor.jpg \
  --out out/market-final.png
```

A LICENSING NOTE

`faceswap` uses the inswapper model, whose weights are **non-commercial** (InsightFace's terms). It is perfect for personal work, series continuity, and studies; check the licence before you put a swapped face on something you sell. `restore-faces`, which only sharpens what is already there, carries no such restriction.

Extending the frame: `outpaint`

Finally, when the poster needs **more canvas** — headroom for a title, a wider lane — **outpaint** pads the image and paints the new border to match, so a 1:1 render can grow into the tall format a poster wants.

● ● ● *Grow the canvas upward for a title*

```
$ plakat outpaint out/market-lane.png --top 320 \  
--prompt "string lights and dark sky above the lane"
```

Edit, don't re-roll

Every tool in this chapter shares one idea: **keep the good render and change only what is wrong**. A soft face, a stray object, a background, a too-square frame — none of these is a reason to abandon an image you spent real effort composing. The instinct to re-roll the seed is usually the expensive instinct. With the poster now edited to clean — hands fixed, clutter gone, the face right — we can move to the two passes that turn a correct render into a finished, printable object: making it feel real, and making it big.

WHAT YOU LEARNED

- Edit the pixels you have instead of re-rolling: `img2img` re-renders guided by the image (keep **strength** low on finished art — whole-image high-strength re-renders tend to make cohesive work worse).
- Fix one region with a `--mask` (inpaint); make the mask by **clicking** the object with `segment`, or paint your own.
- `remove` erases an object and fills the hole — select it by `--point`, `--box`, depth band, or open-vocabulary `--what` "a modern sign".
- `replace-bg` swaps the background while keeping and decontaminating the subject; `restore-faces` sharpens soft faces, and `faceswap` transplants a specific face (non-commercial weights).
- `outpaint` extends the canvas — headroom for a title, a wider frame — painting the new border to match; the through-line is **keep the good render, change only what's wrong**.

CHAPTER 13

13

Making It Look Real

There is a particular sheen that says “made by a model”: too clean, too even, edges a hair too crisp, a plasticky perfection no camera and no printing press ever produced. Your poster may be beautifully composed and still wear that sheen. The `naturalize` pass exists to take it off — to add back the small analog imperfections that make an image read as a **made object** rather than a generation. It is the quiet, final craft step, and it is entirely weight-free: no model, no GPU, just honest signal processing.

The AI fingerprint, and how to dull it

naturalize applies an analog post-pass — the kind of thing that happens to a real photograph on its way through a lens, a sensor, and a print: a whisper of film grain, a touch of chromatic aberration at the edges, a gentle vignette, a little bloom in the highlights, and a desaturating film grade that pulls the colour off its digital saturation.

● ● ● *The finishing pass*

```
$ plakat naturalize out/market-final.png --out out/market-graded.png
  ● detecting medium ... photographic
  ● grade · grain · aberration · vignette · bloom
  ✓ out/market-graded.png
```

TERM **naturalize**

A weight-free analog post-pass that reduces the “AI-generated” fingerprint of an image — film grain, chromatic aberration, vignette, bloom, and a desaturating film grade. It aims at **realism**, not a vintage-filter look, and touches only the finish, never the composition. No model runs; it is fast and deterministic.



Image 07 — after the naturalize pass: the same frame with grain, a gentle grade, and a little bloom. Less rendered, more made.

The result should be subtle. Put the two side by side and the graded one simply looks **less rendered** — the amber glow has a little bloom, the fog carries faint grain, the corners fall off gently the way a real lens darkens them. Nothing about the scene changed; only its surface.

Realism, not a filter

It is worth being precise about what naturalize is **for**, because the name invites a wrong guess. It is not a nostalgia filter that makes everything look like a 1970s snapshot. It is a **de-slop** pass: it targets the specific tells of machine generation and dials them down toward how real optics and real media behave. Ask for “vintage”

and you want a **look** (Chapter 11’s job — a LoRA or a grade). Ask for naturalize and you want the machine-ness **reduced**, whatever the style.

WHY NOT JUST IMG2IMG IT?

It is tempting to run a finished poster through a light `img2img` to “make it more real.” Resist it on cohesive art. A whole-image re-render — even a gentle one — tends to **regress** a carefully composed image: it softens intentional choices and re-introduces its own artefacts. `naturalize` is weight-free precisely so it can improve the **finish** without re-rolling the **content**. Any part of the pipeline that produces smearing should be fixed upstream (model, steps, seed), not papered over with another render.

Protecting the figures

The plain grade above is weight-free and gentle — it will not maul a face. But `naturalize` has a heavier, **model-backed** mode as well: `--repaint` re-paints the image in a chosen medium (oil, watercolour, ink) to build real brush character. That mode **can** disturb faces, so it carries a protection scope that shields the people while it loosens everything else.

● ● ● *A painterly repaint that spares the figures*

```
$ plakat naturalize out/market-final.png \
  --repaint --repaint-protect figures
```

`--repaint-protect figures` (the default when `--repaint` is on) preserves each figure’s whole body — face, hands, clothing — feather-composited back over the repainted surroundings; `faces` protects only the faces; `none` repaints everything (most painterly, but small figures can melt). For a portrait-forward poster, keep the figures protected: the lane, fog, and cart take the painterly texture while the two people stay crisp. `--repaint` needs a model; the plain weight-free grade does not, and is the right default for a photographic poster like ours.

Naturalize from the prose

Because finishing is part of making an image, plakat can bake a scene-tuned naturalize into the compile itself — the analyzer’s companion, `--make-composition`, even suggests one. In prose it is a directive, so the finish travels with the scene.

● ● ● *A finish that travels with the scene*

```
name: market-lane
naturalize: grain=0.3 vignette=0.2 desaturate=0.15
A night market lane, a vendor at his cart, a customer leaning in.
```

Now every render of this scene emerges already graded, with the figures protected — no separate command, no forgotten step. For a poster you will render many times, that consistency is worth having; the finish becomes part of the source, like everything else in the prose project.

YOU WILL SEE IT WORKING

A naturalize pass prints progress as it runs — detecting the medium, then each stage of the grade — rather than sitting silent. On a large print-resolution image the pass takes a moment; the running messages tell you it is working, not stuck.

A correct image becomes a finished one

With the poster graded, it has crossed the line from **correct** to **finished**: it no longer announces that a model made it. Two practical steps remain before it is a print — making it physically large enough, and signing it so its origin travels with it. Those are the last chapter of the making.

WHAT YOU LEARNED

- `naturalize` is a **weight-free** analog post-pass — grain, chromatic aberration, vignette, bloom, a desaturating grade — that reduces the “AI-generated” fingerprint without touching the composition.
- It aims at **realism**, not a vintage filter: it is a **de-slop** pass that dials the machine-tells toward how real optics behave; a “look” is still Chapter 11’s job.
- Do **not** finish cohesive art with a whole-image `img2img` — it regresses careful work; `naturalize` improves the finish precisely because no model re-renders the content.
- The heavier **model-backed** `--repaint` mode builds real brush character; `--repaint-protect figures|faces|none` shields the people while it works — but the plain weight-free grade is the right default for a photographic poster.
- Bake it into the prose with a `naturalize:` directive so the finish travels with the scene and every render emerges already graded.

CHAPTER 14

14

Bigger, and Sharper

An SDXL poster is born at 1024 pixels square. That is a fine size for a screen and far too small for a print — a B2 poster wants thousands of pixels on a side. The last technical step is to enlarge the image to print dimensions without it going soft. plakat offers two roads: a fast classical resize that just makes the pixels bigger, and a diffusion upscaler that **invents** believable detail as it grows. Which you choose depends on how far you are enlarging and how close a viewer will stand.

The fast road: classical upscale

The default `upscale` uses a high-quality classical filter (Lanczos). It is instant, deterministic, and adds no new detail — it interpolates what is already there. For a modest enlargement, or when you simply need more pixels for layout, it is exactly right.

● ● ● *A clean classical enlargement*

```
$ plakat upscale --in out/market-graded.png --scale 2 \  
  --out out/market-2x.png  
✓ 10242 → 20482 (Lanczos)
```



Image 08 — the graded frame enlarged to print resolution, its detail intact at the size a viewer will actually stand from.

Push a classical upscale too far — 4× and beyond — and the image softens: there is only so much a filter can do with detail that was never captured. That is where the second road comes in.

The detailed road: diffusion upscale

upscale --diffusion enlarges the image **and** hallucinates plausible fine detail as it goes — cobblestone texture, the grain of the cart's wood, threads in the vendor's apron — by running a tile-based diffusion pass guided by the original. It is slower and it invents, but for a large print viewed close it is the difference between soft and crisp.

● ● ● *Enlarge and add detail*

```
$ plakat upscale --in out/market-graded.png --diffusion --scale 4 \
  --out out/market-print.png
↳ tiling · ControlNet-tile guidance · colour-matched seams
✓ 10242 → 40962
```

TERM Diffusion upscale

A tile-based super-resolution pass (`upscale --diffusion`) that enlarges an image while a diffusion model, guided by the original via a tile ControlNet, invents believable high-frequency detail. Seams between tiles are colour-matched and the input pre-sharpened so the result is seamless. Slower than classical, and it adds detail rather than merely interpolating.

GUIDED, NOT REINVENTED

Because the diffusion upscale is **guided by your image** tile by tile, it adds detail without wandering off your composition — unlike a naive whole-image `img2img` at high strength. It is the one place a diffusion re-pass is the right tool on a finished poster, precisely because it is constrained to enlarging what is already there.

Faces survive the enlargement

Enlarging can expose a face that was fine at 1024² but soft at 4096². Pair the upscale with the face restorer from Chapter 12 — upscale first, then restore — so the faces are refined at the size they will actually be seen.

● ● ● *Upscale, then sharpen the faces*

```
$ plakat upscale --in out/market-graded.png --diffusion --scale 4 \
  --out out/market-print.png
$ plakat restore-faces out/market-print.png
```

How big is big enough?

A quick rule keeps you from over- or under-cooking the enlargement. Prints are usually reckoned at 300 DPI, so multiply the physical size in inches by 300 to get the pixels you need on each side.

A4 / Letter	~2500 × 3500 px. A 2–3× upscale from 1024 ² covers it.
A3 / Tabloid	~3500 × 5000 px. A 4× diffusion upscale, then restore faces.
A2 / B2 poster	~5000 × 7000 px. 4× diffusion upscale of a portrait-outpainted render, viewed from a step back.

ORDER MATTERS AT THE FINISH

Do the finishing steps in the right sequence: compose and edit at native size, **naturalize**, then **upscale**, then **restore faces** on the enlarged image. Grade before you enlarge (so the grain scales with the image), and restore faces after (so you sharpen the pixels a viewer will actually see). Getting the order wrong gives you grain that looks digital or faces that soften again on enlargement.

A poster at print size

Our NIGHT MARKET is now a large, crisp, print-ready image: composed over Part IV, graded in the last chapter, enlarged here to poster dimensions with its detail intact and its faces sharp. It is, as a **picture**, done. One thing separates a finished picture from a finished **poster** you can hand to a printer or a client: proof of where it came from, its recipe for remaking, and the title type across the top. That is the final chapter.

WHAT YOU LEARNED

- Renders are born near a model's native size (SDXL at 1024^2) and **upscaled** to print dimensions — never rendered huge directly.
- Classical **upscale** (Lanczos) is instant and interpolates existing detail — right for modest enlargements; it softens past 4×.
- **upscale --diffusion** enlarges **and** invents believable detail via a tile ControlNet, with colour-matched seams — the choice for large prints viewed close, and a **guided** re-pass that won't wander off your composition.
- Enlarge **then** **restore-faces**, so faces are sharpened at the size they'll be seen; size to 300 DPI × the physical inches you need.
- Finish in order: compose → naturalize → upscale → restore faces, so grain scales correctly and faces stay crisp.

CHAPTER 15

15

The Title, the Signature, and the Export

A finished picture is not yet a finished **poster**. A poster has a title across the top; it carries proof of where it came from and a recipe for remaking it; and it ships in the right file for wherever it is going. This last chapter of the making adds the type, signs the work, and sends it out — and it is honest about the one thing plakats should **not** be asked to do.

The title: type belongs in a layout, not a prompt

You may be tempted to ask the diffusion model for the words NIGHT MARKET across the sky. Don't. Diffusion models are weak at rendered text — they produce convincing **shapes** of letters that spell nothing, or “NIGTH MARKPT,” and no amount of prompting reliably fixes it. The professional move is the one a real poster designer makes: render the **image** with **plakat**, and set the **type** in a layout step, where a font is a font.

plakat's **compose** command is that layout step. It stacks image layers — a background, placed cut-outs, a title bar — with z-order, position, scale, and opacity, on the CPU, no diffusion involved.

TERM **compose**

A layered-composition command: `plakat compose layout.hjson` stacks image assets — a rendered background, cut-out subjects, a title graphic — with per-layer position, scale, z-order, and opacity. No model runs; it assembles finished pixels. It is where poster **type** and any hand-made graphic elements go, kept separate from the diffusion render.

● ● ● *Compose the poster: art + title*

```
# layout.hjson
{
  size: 4096x5600
  layers: [
    { image: out/market-print.png z: 0 fit: cover }
    { image: assets/title-night-market.png z: 1
      x: 0.5 y: 0.08 scale: 0.8 anchor: top-center }
  ]
}

$ plakat compose layout.hjson --out out/poster.png
✓ 2 layers → out/poster.png
```



Image 09 — the portrait poster crop, dark sky held clear at the top where the title type will be set in the layout step.

The title graphic itself — the words NIGHT MARKET in a chosen typeface — you make in whatever tool sets type well (including plakat’s own **bookart**, which draws clean glyph-based lettering for exactly this). The point is separation: the model paints the scene; type is placed as type. Your poster gets crisp, correct letters, every time.

WHY THIS IS THE HONEST CHAPTER

A book that promised “just prompt the model for your title” would be selling you a frustration. plakat is a studio, not a single magic box: it renders the picture superbly and hands the lettering to a layout step that does **that** job well. Using each tool for what it is good at is the whole philosophy of the program.

Signing the work: provenance etching

A poster that leaves your machine should carry proof it came from you. `plakat` can **etch** a provenance mark into any image it produces — an invisible, multi-layer identifier that survives ordinary handling — with a single global flag.

● ● ● *Etch provenance on the way out*

```
$ plakat compose layout.hjson --out out/poster.png --etch
↳ etched EtchId (manifest · pixel · fingerprint layers)
```

TERM Provenance etching (`--etch`)

An opt-in mark written into images `plakat` creates: a 64-bit identifier carried across several layers (a metadata manifest, a pixel-domain watermark, a perceptual fingerprint). `plakat doctor --if-plakat IMAGE` reads the surviving layers back into a graded verdict on whether an image originated from `plakat`. Honest about its limits — it is not removal-proof, and absence of a mark is not proof of forgery.

Later — on your own image or someone else's — you can ask whether it came from `plakat`, and get a graded answer rather than a guess.

● ● ● *Verify an image's origin*

```
$ plakat doctor --if-plakat out/poster.png
verdict: LIKELY plakat (manifest ✓ · fingerprint ✓)
```

The recipe travels with the image

Every `plakat` render already carries its own recipe — the metadata sidecar and the in-PNG parameters block from Chapter 1. At the end of a project that pays off twice.

metadata reads a poster's settings back out of the file; **clone** turns them into a runnable command, so months later you can reproduce or fork the exact render.

● ● ● *Recover how it was made*

```
$ plakat metadata out/market-print.png
  model sdxl · seed 1000 · 30 steps · euler-a · 10242 · loras: ink-
poster-xl
$ plakat clone out/market-print.png
  plakat generate "night market lane, ..." --model sdxl --seed 1000 \
    --steps 30 --scheduler euler-a --loras ink-poster-xl:0.65
```

For a **series** of posters, **gallery** builds a browsable Markdown index of a whole folder — each thumbnail with its prompt and settings — from the metadata alone. It is the reproducible contact sheet for a body of work.

● ● ● *Index the series*

```
$ plakat gallery out/ --out out/index.md
  ✓ 6 images · thumbnails + per-image recipe → out/index.md
```

Choosing the file

Last, the container. PNG is the default and the right choice for a master and for anything that will be re-edited or uploaded — it carries the parameters block that other tools read. WebP ships perceptually-equivalent files about a third smaller for the web, at the cost of that embedded block (the JSON sidecar still travels alongside, so **metadata** and **clone** keep working). Master in PNG; export a WebP for the web when size matters.

Start to finish

Trace the whole arc. We installed plakat and made a single lantern in the fog. We built a prose project and wrote the scene in sentences. We **analyzed** it before spending a render, **fixed** its safe problems, and let the smysl corpus remember why. We compiled, chose SDXL, and tuned the frame. We added a second figure,

separated the cart, related the two people, dressed the image in a look, and pinned the vendor's face for a series. We edited the last flaws, naturalized away the machine sheen, enlarged to print, set the title in a proper layout, signed the work, and exported it. One poster, start to finish — and every command you reached for, reached for in the order the work asked.

That is the whole of it. What follows is reference: the directives, the models, and a one-page map of the polish loop, for when you are making your **own** poster and want the lever without the story.

WHAT YOU LEARNED

- Set poster **type** in a layout, not a prompt: diffusion models garble text, so render the image with `plakat` and place the title as its own layer with `compose`.
- `--etch` signs an image with an invisible, multi-layer provenance mark; `doctor`
`--if-plakat` reads it back into a graded origin verdict (honest about its limits).
- Every render carries its recipe: `metadata` reads it back, `clone` turns it into a runnable command, and `gallery` indexes a whole series from the metadata alone.
- Master in **PNG** (it carries the parameters block); export **WebP** for the web when file size matters — the JSON sidecar keeps `metadata` / `clone` working either way.
- The arc complete: `install` → `prose` → `analyze` → `fix` → `compile` → `tune` → `compose` → `style` → `edit` → `naturalize` → `upscale` → `title` → `sign` → `export` — one poster, each tool used for what it does best.

CHAPTER 16

16

Layered Generation

You have taken one poster from a blank project to a signed print, and along the way you met the composition tools the compile path gives you: **foreground** to name the figures, **objects:** to keep a cart from melting into its vendor, **relate:** to make two people face each other. Those staged NIGHT MARKET beautifully — two figures, one prop, one interaction. This chapter is about the scene those tools **can't** quite hold, and a second engine built for it.

The ceiling is always the same one: **fusion**. A diffusion model paints the whole canvas at once, so the more independent subjects you ask for, the more it blends them — three distinct people become a two-and-a-half-person smear, each prompt

bleeding into its neighbour. **objects:** and **relate:** push that ceiling up a little. Layered generation moves it somewhere else entirely: instead of asking the model to keep several subjects apart **while** painting them, it draws each subject **alone**, then paints the final image once with those solo drafts holding everyone in place.

The idea: constraints, not pixels

A layered render runs in three moves.

First it splits your scene into a **plan**: a backdrop (the environment) plus a small set of independent **subject layers**, each with its own full-detail prompt and a box on the canvas. Second, it **drafts** each layer alone — one subject per image, where binding is trivial because there is nothing to fuse with — and composes those drafts into a single low-resolution **guide**. Third, it runs **one ordinary denoising trajectory** of your finish model, gently steered toward the guide’s low frequencies inside each subject’s box during the early steps, then left free to paint its own detail.

THE ONE RULE THAT MAKES IT SAFE

Layers are **constraints on layout**, never pixels copied to the output. Not one pixel of a draft survives into the final image — only the coarse, low-frequency answer to “roughly what goes where” is borrowed, and only early, and only inside each box. The finish is still a single, coherent render in your model’s own hand. That is why a layered image looks painted, not pasted.

The plan

A plan is a small HJSON file. **plakat layers new** scaffolds one to edit; here is the one this chapter renders, a busier NIGHT MARKET than the poster dared — a vendor, a customer, and a glowing lantern, three subjects that plain prose would fuse.

● ● ● corpus/layered/night-market.hjson

```
size: "1216x832"
global: {
  palette: "warm amber and honey lantern glow, soft teal night shadows"
  light: "warm lantern light filling the lane, soft glow on wet
stones, gentle fog"
  medium: "cinematic poster illustration, warm and atmospheric" #
FINISH only
}
prompt: "a lively night market lane glowing with warm lantern light, a
vendor
        serving a customer, stalls and paper lanterns receding into
soft fog"
backdrop: { prompt: "a lively night market lane, wet cobblestones
catching warm
        lantern light, rows of glowing stalls and lanterns
into fog",
weight: 0.55, window: 0.30 }
layers: [
  { id: "vendor", prompt: "a food vendor in a dark apron leaning over
a cart", box: [0.08, 0.34, 0.46, 0.95], depth: 0.30, weight: 0.78, window:
0.40 }
  { id: "customer", prompt: "a customer in a warm coat reaching for a
paper cup", box: [0.52, 0.36, 0.88, 0.96], depth: 0.35, weight: 0.78,
window: 0.40 }
  { id: "lantern", prompt: "a large round red paper lantern, glowing",
box: [0.66, 0.10, 0.86, 0.34], depth: 0.60, weight: 0.70, window: 0.34 }
]
draft: { model: "sdxl", seed: 11 }
```

Each layer's **box** is `[x0, y0, x1, y1]` in fractions of the canvas (a **place:** phrase like "center-left mid" works too, if you would rather describe than measure). **depth** orders the layers front-to-back — **0** is nearest, **1** farthest — so a nearer subject wins where boxes overlap. The **global** look is anchored into every draft **except** **medium**: technique is the finish's job, so only palette and light travel down to the solo drafts.

The **weight** and **window** on each layer are the **anchor strength** — how hard, and for how long, the finish is pulled toward that subject's place in the guide. We will come back to them at the end of the chapter; they are the dial between "follow the plan exactly" and "blend into one cohesive scene."

TERM box / place / depth

A layer's place in the frame. `box: [x0,y0,x1,y1]` gives an explicit rectangle in `[0,1]`; `place: "center-right mid"` resolves the same from words. `depth` (0 near ... 1 far) sets the front-to-back order the drafts are composed in. Together they are the **layout** the guide will hold — the constraint, not the content.

Size classes: what the anchor can hold

Not every box is big enough to steer. `plakat layers lint` validates the plan and reports, for your finish model, which class each layer falls in.

● ● ● *Lint the plan for SDXL*

```
$ plakat layers lint corpus/layered/night-market.hjson --model sdxl
◆ lint ... (sdxl · 1216×832 px · 3 layer(s))
  · layer "vendor":   anchored (462px shorter side)
  · layer "customer": anchored (438px shorter side)
  · layer "lantern":  anchored (200px shorter side)
→ PASS — clean
```

An *anchored* layer is large enough to get the full treatment: a guide anchor, plus the verify-and-repair pass later. A *hinted* layer is smaller — named in the finish prompt and checked, but not anchored. A *lifted* layer is tiny — too small to steer, so its structure comes from a dedicated crop-and-regenerate pass. All three subjects here are comfortably anchored. `layers show` prints the same resolution and draws the boxes, coloured by class, so you can **see** the layout before spending a single GPU-second:

● ● ● *Draw the plan (offline — no model)*

```
$ plakat layers show corpus/layered/night-market.hjson \
  --model sdxl --boxes plan.png
```

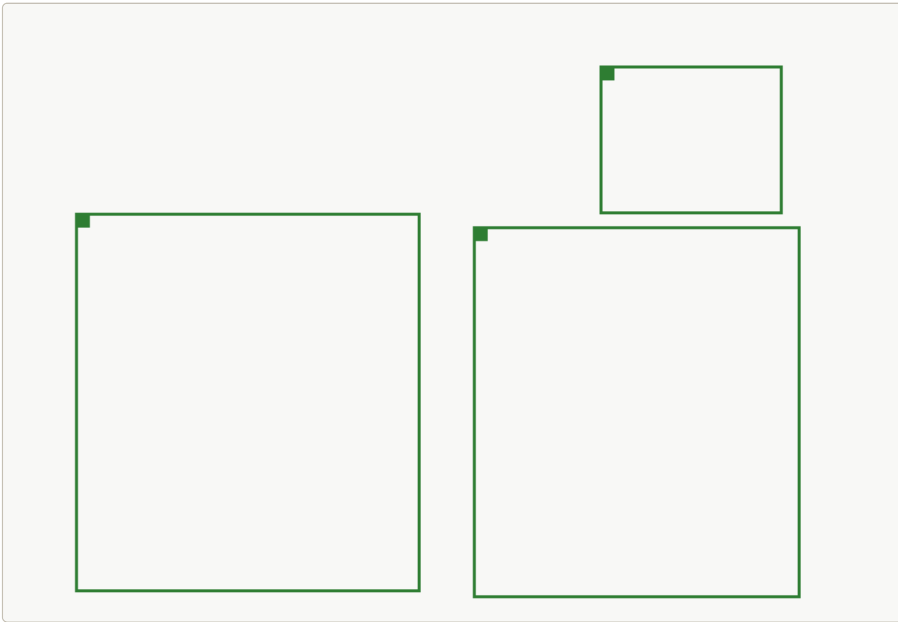


Image 11 — the plan, drawn: three anchored subject boxes on the canvas (vendor lower-left, customer lower-right, lantern upper-right). This is pure geometry — no diffusion — so it is the cheapest way to check a composition before you render it.

Rendering

With the plan lint-clean, one command runs the whole pipeline — drafts, guide, and the anchored finish.

● ● ● *The full layered render*

```
$ plakat layers render corpus/layered/night-market.hjson \
  --model sdxl --draft-model sdxl --draft-steps 8 --steps 36 \
  --guidance 6.5 --ramp 0.2 --seed 11 --keep stages/ --out poster.png
```

The finish model is SDXL — the book’s model. `--draft-model` picks the model that draws the solo drafts; the drafts only need to be **roughly** right — their high-frequency detail is thrown away — so a few steps of SDXL are plenty (`--draft-steps 8`). `--ramp` softens how the anchor lets go (more on that below). `--keep stages/` writes the intermediate drafts, the composed guide, and

the weight/window anchor maps next to the output, which is worth doing the first few times so you can watch the machine think.



Image 12 — the finished layered render: a cohesive night-market lane, its lantern-strung eaves receding into real perspective, the vendor at her cart on the left and the crowd browsing the stalls on the right, all painted together in a single SDXL pass. Three planned subjects that plain prose would have fused into a smear, woven into a place with genuine depth — not pasted onto a void. (Rendered with the structure and self-correction controls below.)

FLUX WORKS TOO

Layered generation is wired for the SD family (SD 1.5 / SDXL) and Flux. Point --model at a Flux alias and the guide is encoded in Flux's own latent space — the anchor is the same, only the finish family changes. (SD3, Sana, PixArt and Cascade finishes are on the roadmap.)

Cohesion: the anchor is a dial

Look again at that render. The vendor, the customer and the lantern are where the plan put them — but they are not pasted onto a black void; they sit in a real lane, with stalls, wet cobblestones and strung lanterns tying the scene together. That

cohesion is not automatic, and the naive version of this plan gives you the opposite: three well-drawn subjects floating in the dark. Two things produce the difference.

The first is the **backdrop**. Each subject is drafted ALONE on a plain surround (so it mattes cleanly), which means the only thing joining the subjects in the guide is the backdrop draft. A thin, dark “night lane” backdrop makes the guide islands-in-black, and the finish faithfully paints islands in black. Give the backdrop a real, lit environment — “wet cobblestones catching warm lantern light, rows of glowing stalls” — and the finish has something to weave the subjects into.

The second is **anchor strength** — each layer’s **weight** (how hard) and **window** (how long), plus the **--ramp** that softens the hand-off. Turn them up and the finish reproduces the guide’s layout exactly, subjects crisp but disconnected. Turn them down and the finish treats the guide as a loose suggestion, blending the subjects into a scene of its own painting. The plan above uses **weight: 0.78, window: 0.40** on the figures — down from the firm default — and renders with **--ramp 0.2**: enough to keep the vendor at his cart on the left and the lantern up-right, loose enough to let SDXL fill the lane between them.

The guide also helps the seam automatically, so subjects sit **in** the scene rather than on it: each matted subject is colour-harmonised toward the backdrop’s palette (**--harmonize**), and a soft contact shadow is laid under it (**--no-ground** turns this off) — which is why the figures in Image 12 cast shadows onto the wet cobblestones instead of hovering above them.

THE TRADE YOU ARE MAKING

Anchor strength is the dial between **layout fidelity** and **cohesion**. High weight / long window = subjects land exactly where planned, at the cost of looking composited; low weight / short window = one seamless scene, at the cost of the plan being a suggestion. If a render looks like a collage, lower the weights and enrich the backdrop; if a subject drifts out of its box, raise them. There is no universally right setting — only the one that suits the image you are making.

Structure and self-correction: `--depth-control` and `--adapt`

The anchor holds **layout** — where each subject sits — but only in the low frequencies, so it says little about a subject’s **structure**: the shape of the cart, the fall of a coat, the architecture receding down the lane. Two opt-in controls sharpen the finish beyond the layout.

`--depth-control` runs a depth estimator over the composed guide and drives a ControlNet-Depth pass on the finish alongside the anchor. The guide already knows where everything is; depth turns that into real geometry — the subjects gain form and the scene gains perspective, **without copying a single pixel** (a depth map is a constraint, exactly like the anchor). It is SD-family only for now, and applied early-to-mid (`--depth-end`), so it shapes the composition and then lets the finish paint its own detail.

`--adapt` closes the loop. After the finish, it verifies each subject (the detector below) and, for any that drifted, **raises that layer’s anchor and re-renders at the same seed** — so the subjects that landed stay exactly put while the strays are pulled in harder. It is a cleaner fix than the masked repair below, because the subject is re-painted **in context** rather than patched in afterward; the two compose, `--adapt` first.

● ● ● *The full render, with structure + self-correction (produced Image 12)*

```
$ plakat layers render corpus/layered/night-market.hjson \
  --model sdxl --draft-model sdxl --draft-steps 8 --steps 36 \
  --guidance 6.5 --ramp 0.2 --seed 11 \
  --depth-control --adapt \
  --keep stages/ --out poster.png
```

A NOTE ON VERIFYING PAINTED SUBJECTS

The detector that drives `--adapt` (and `--repair`) is trained on **photographs**, so it scores a painted or illustrated figure far lower than a photographic one. Layered renders relax the detection threshold automatically for a non-photographic medium, but a stylized scene can still read as a phantom miss — a subject that is plainly **there** going undetected. When that happens, `--adapt` simply spends its rounds and stops; the image is unharmed. Trust your eyes over the verdict on painterly work.

Did it land? Verify, repair, lift

The layered pipeline can **check its own work**. `plakat layers verify` asks an open-vocabulary detector (OWL-ViT) whether each anchored subject actually rendered inside its box, and exits non-zero if any missed — so it can gate a repair.

● ● ● *Verify each subject is where the plan put it*

```
$ plakat layers verify corpus/layered/night-market.hjson \
  --image poster.png --model sdxl
✓ vendor    anchored score 0.14 iou 0.71 "a food vendor ..."
✓ customer  anchored score 0.12 iou 0.68 "a customer ..."
✗ lantern   anchored score 0.00 iou 0.00 "a large red paper
lantern"
→ FAIL — 1 missing: lantern
```

When a subject misses, `plakat layers repair --auto` verifies first and then re-asserts just the layers that failed, with a masked `img2img` pass over each one's box — the rest of the image is preserved. And for a subject too small to anchor in

the first place, `plakat layers lift` crops its box, regenerates it at a workable resolution, and composites it back with a feathered seam. Both are targeted touch-ups, not a full re-render.

Diff: reading the anchor

`plakat layers diff` compares two images at the plan's low-frequency band — the band the anchor actually constrains — and reports, whole-canvas and per box,

how closely they agree. Point it at the guide and the finish to **measure** whether the anchor took:

● ● ● *How well did the finish track the guide?*

```
$ plakat layers diff corpus/layered/night-market.hjson \
  --a stages/__guide.png --b poster.png --model sdxl -o heat.png
```

It is model-free and instant, and the heatmap it writes shows exactly where the finish drifted from the plan — a fast, honest read on a render you are not sure about.

Skippping the hand-written plan: the planner

You do not have to author the HJSON yourself. `plakat layers plan` hands a prose description to an LLM and gets a plan back — global look, backdrop, and separated subject layers with placements — ready to lint, edit, and render.

● ● ● *Prose → plan, via a local model*

```
$ plakat layers plan \
  "a night market: a vendor at a cart, a customer ordering, a lantern
above" \
  -o market.hjson --model sdxl --provider ollama:qwen2.5-coder:14b
```

`--provider` chooses the LLM: omit it for the small in-process model, or point it at a model you have pulled with Ollama (`ollama:qwen2.5-coder:14b`) for sharper decompositions — a bigger model places the subjects on opposite sides of a “face-off” instead of stacking them. Whatever you get is an ordinary plan file: read it, nudge a box, and render.

Layering from prose: compile --layered

Everything in this chapter so far has started from a **plan** — hand-written, or from `layers plan`. But you have spent the whole book in the **compile** path: prose in a `.txt`, `plakat compile` out to a scenario. That path can reach layered generation on its own.

Add `--layered` to any compile and it inspects each scene it resolves. When a scene has **two or more independent foreground figures that do not interact** — exactly the fusion-prone case — compile decomposes it into a layered plan automatically and points the scenario at it. There is nothing new to write in the prose: you already name the heroes with `foreground:` (Chapter 9), and `--layered` simply turns that figure list into subject layers.

● ● ● *Compile prose straight to a layered scenario*

```
$ plakat compile market.txt --layered --out market.hjson
✓ compiled → market.hjson
✓ layered plan → a_bustling_night_market_lane_glowing.layered.hjson
```

The scene that qualified becomes a `type: layered` task, pointing at a sidecar plan written next to the scenario:

● ● ● *market.hjson — the layered task*

```
tasks:
[
  {
    name: a_bustling_night_market_lane_glowing
    type: layered
    layered: { plan:
"a_bustling_night_market_lane_glowing.layered.hjson" }
  }
]
```

That sidecar is an ordinary plan — the same HJSON you met at the top of this chapter, built for you: the backdrop from the scene’s prose, one placed subject layer per `foreground` figure, the finish look from your `style:`. Read it, nudge a box,

plakat

and render it with `scenario market.hjson` like any other file. A scene with a single hero — or two figures the prose says are interacting, a `relate:` contact — is left on the one-pass compile path untouched; `--layered` only reroutes the scenes that actually need it.

--IMPROVE SHARPENS EACH LAYER ON ITS OWN

The polish loop understands layers too. Run `compile --layered --improve` and, for a layered scene, `--improve` optimises **each layer's prompt separately** — rendering and aesthetically scoring one subject at a time, keeping the best wording per layer — rather than scoring the whole crowd at once. Each layer keeps its own history in the `smysl` corpus, so `--improve-skip-good` gates them one by one, and the winning prompts are written straight back into the sidecar plan.

Which road? Layered vs. the compile path

Two engines now stage a scene, and they are not rivals — they are for different scenes.

PICK THE TOOL FOR THE NUMBER OF SUBJECTS

For one to three figures with a clear interaction — the whole of NIGHT MARKET — the compile path (`foreground`, `objects:`, `relate:`, `control-generate`) is the right road: it is simpler, it is what the rest of this book uses, and it renders in one pass. Reach for **layered** when a scene has **several independent subjects that each need full detail** and keep fusing no matter how you phrase them — a market of distinct stalls, a group portrait where every face matters, a poster with a cast. When `compile --analyze` flags fusion you cannot phrase your way out of, layers are the next rung — and `compile`

`--layered` is the bridge that climbs it without ever leaving the prose path.

That is the last engine in the book. You have every tool a poster needs — from a single prose line to a self-checking, layer-by-layer composition — each for the job it does best.

WHAT YOU LEARNED

- Diffusion **fuses** independent subjects; layered generation avoids it by drafting each subject **alone**, then painting the final image once with those drafts as a guide.
- Layers are **constraints on layout, never pixels** — only the coarse low frequencies of the drafts steer the finish, early and inside each box, so the result is painted not pasted.
- A **plan** is a small HJSON: a **global** look, a **backdrop**, and subject layers with a **box / place** and **depth**; `layers lint` reports each layer's size class (anchored / hinted / lifted) and `layers show` draws the boxes offline.
- `layers render` runs drafts → guide → one anchored finish (SD family / Flux); `--keep` saves the stages. `verify` (OWL-ViT) checks each subject
`repair`
`landed`, `--auto` fixes the misses, `lift` rebuilds tiny subjects, `diff` measures the anchor.
- **Cohesion is a dial**: a rich, lit **backdrop** gives the finish an environment to weave the subjects into, and each layer's **weight / window** (with `--ramp`) trades layout fidelity for a seamless scene — lower them and enrich the backdrop if a render reads as a collage.
- **Structure + self-correction**: `--depth-control` drives a ControlNet-Depth pass off the guide so subjects gain form and the scene gains perspective (a depth map is a constraint, not pixels); `--adapt` re-renders a drifted subject at a stronger anchor before any masked repair. On painterly work, trust your eyes over the photo-trained detector's verdict.
- `layers plan "<prose>"` decomposes a description into a plan via an LLM (`--provider ollama:<model>` for a bigger local model). Reach for layered generation when **many** independent subjects keep fusing — otherwise the compile path is simpler.
- The **compile path reaches layered on its own**: `compile --layered` decomposes any fusion-prone scene (two or more independent **foreground** figures, no interaction) into a **type: layered** task plus an auto-built sidecar plan, and `compile --layered --improve` then sharpens **each layer's prompt separately**.



PART VI

Reference

APPENDIX A



The Prose Directive Reference

This is the working reference for the prose you write in a `prompts.txt` — every directive the book used, grouped by what it controls, plus the in-prose syntax for weights, scheduling, and structure. Directives are `key: value` lines; free text between them is the scene's prose. Global-block directives set project defaults; most can be overridden per scene.

Project & output

model:	Model alias or HF repo (sd15, sdxl, flux-schnell, ...). Global; a scene may override for itself.
size:	Render size, e.g. 1024x1024. Prefer the family's native square.
seed:	Base seed. Per-image seeds step from it; fix it for reproducibility.
count:	Images per task.
out:	Output directory (created on first run).
enhancer:	Enhancement provider: local · deepseek · gemini · auto.
device:	auto · metal · cuda · cpu.

Sampler & quality

steps:	Denoising steps (SDXL: 28–35 is plenty).
guidance:	Classifier-free guidance (~7 a good middle; 0 for turbo).
scheduler:	default · ddim · euler-a · unipc.
clip-skip:	1 (default) or 2 (SD 1.5/2.1 community checkpoints).
negative:	Scene-specific terms to avoid, added to the automatic quality set.

Reuse & structure

name:	The scene's name (becomes the task name and output stem).
skip:	true to skip a scene without deleting it.
component.<n>:	A named, reusable scene fragment. Reference it from a DIRECTIVE (below), not by bare mention. Global or scene-local (local overrides).

composition:	Comma-list of components joined in order, BEFORE the free-text prose (compose-then-prose).
scene.<n>: weather.<n>:	/ Named variant axes; each render picks one of each.
loras:	LoRA stack, name:strength (e.g. ink-poster-xl:0.65).
lora-trigger:	Trigger word(s) a named LoRA needs to activate.
@include path	Inline another prose file (with optional key=value passing).

Composition & control

foreground:	Comma list of intended focal figures; fixes the figure count.
objects:	Comma list of props that must render SEPARATELY from figures (the cart).
relate:	A relationship between two components: <a> <verb> . Placement verbs (behind, beside, facing) stage layout; ATTACH verbs (holding, wearing, carrying) fold b INTO a's description so a held prop doesn't float.
control-generate:	Enable the composition pass; value names the draft model (sdxl/sd15).
control-generate-size:	Draft resolution (match the draft model: 1024 ² for sdxl, 512 ² for sd15).
control-generate-regional:	true (default) paints each figure in its own area; false blends more.
control-generate-figure-poses:	One posture per foreground figure (standing, leaning, sitting...).
control-generate-figure-contacts:	Limb-level touches between figures (arm-around, holding-hands...).

Finishing

naturalize: Scene-tuned finishing pass baked into compile (grade, grain, vignette, repaint-protect=figures).

In-prose syntax

These live **inside** the free-text prose, not as **key:** lines.

(phrase:1.3)	Attention weight — above 1.0 emphasises, below 1.0 eases back. Preserved through compile.
BREAK	Encode what precedes and follows independently, so distinct elements don't bleed.
[a:b:when]	Prompt scheduling — swap a for b once the render passes the fraction `when`.
[a b]	Alternate between a and b each step.

The commands that act on prose

compile FILE	Prose → scenario. --out, --explain, --dry-run, --check, --diff, --watch.
compile --analyze	Feasibility report (grade + risks + fixes); renders nothing.
compile --analyze --fix	Apply safe fixes, back up to FILE.N, record the smysl corpus.
compile --trace "..."	Explain why a phrase is in — or gone from — the prompt (no render).
compile --smysl	Also write the provenance sidecar beside the scenario.
compile --make-composition	Suggest a generation strategy + a cleaner prose rewrite.
scenario FILE	Render every task in a compiled scenario.

THE ONE HABIT

Edit the prose; treat the scenario as compiled output. Every directive here lives in a file you write and read, so `--analyze`, `--fix`, and `--trace` can reason about your intent — which is the whole reason the prose project exists.

APPENDIX B

B

Models, Families, and Budgets

A model is chosen by **family** first — the family sets how you prompt, the native size, and the token budget — and by the specific alias second. This appendix is the map: what to reach for, and what to expect from it. See the live list any time with `plakat models aliases`, and what your machine can run with `plakat doctor --capability`.

The families at a glance

SD 1.5	512 ² native · ~77-token CLIP · keyword-friendly. Small, fast, token-free; the CPU-friendly learning default. Hard text limit.
SD 2.1	768 ² native · OpenCLIP-H · v-prediction. Like 1.5, a step larger.
SDXL	1024 ² native · dual CLIP · ~150 tokens. The reliable general-purpose family; best pose/style adapter ecosystem. The book's choice.
SD 3.x	1024 ² · T5-XXL + dual CLIP · ~256 tokens · strong prose. MMDiT; gated (HF_TOKEN). Low-mem mode fits Medium on 24 GB.
Flux	Transformer · no CLIP token cap · write flowing prose. schnell 4-step (Apache); dev larger and gated. Quantised GGUF for modest RAM.
Cascade	Würstchen-based · ~120 tokens · efficient latent space.
Sana / PixArt	Efficient large-context prose transformers (Gemma-2 / T5-XXL) · ~256 tokens · light on memory.

Common aliases

sd15, sd-1.5	SD 1.5 base (community mirror). Token-free.
sd15-inpaint	9-channel inpainting UNet for masked edits.
sd21	SD 2.1, 768 ² native.
sdxl	SDXL base 1.0 — the poster's model.
sdxl-turbo	Distilled SDXL: 1–4 steps, guidance 0.
pony	Pony Diffusion v6 XL — an SDXL fine-tune (SDXL budget). Pairs with --look pony.
sd35-medium sd35-large	/ SD 3.5, gated. Large needs ≥32 GB or PLAKAT_SD3_LOWMEM=1.

flux-schnell / Flux 4-step (Apache) / larger gated dev; GGUF variants for low
flux-dev RAM.

sana, **sana-1.5,** Efficient prose transformers; T5-scale budget.
pixart

The budget, keyed to the model

The token budget is not cosmetic — it is how much of your prose survives to the render. plakat fits the prompt to the **actual** model you chose, and it gets the aliases right: **pony** is budgeted as SDXL, **pixart** and **sana** as T5-scale, not as a small default. When a prompt runs over, the model-free packer drops generic filler first and never the subject, recording what it cut (Chapter 8).

~77 tokens SD 1.5 / SD 2.1 — front-load the subject; the tail is at risk.
~120 tokens Cascade.
~150 tokens SDXL / SDXL-turbo / pony.
~256 tokens SD 3.x / Sana / PixArt — room for rich prose.
no CLIP cap Flux — write as much flowing prose as the scene wants.

Native sizes

512² SD 1.5, PixArt-512, Sana-512, sd15-inpaint.
768² SD 2.1.
1024² SDXL, pony, SD 3.5, Flux, Sana, PixArt — render here, upscale later.

Choosing, in one breath

Learning / CPU / speed sd15 — small, token-free, forgiving.

Reliable general work `sdxl` — the book's default; best adapter support.

Rich prose comprehension `sd35-medium` or `flux` — big budgets, strong text following.

Fast drafts `sdxl-turbo`, or any base + `--fast` (`lightning/hyper/lcm`).

Stylised character art `pony` — an SDXL fine-tune; pair with a look.

LET THE MACHINE NARROW IT

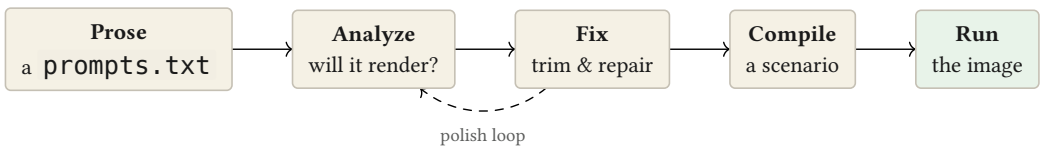
Don't guess what fits: `plakat doctor --capability` judges each model runs / tight / won't-fit on your hardware and names the lever (low-mem mode, quantised GGUF) that rescues a tight one. `plakat models recommend` and `plakat models aliases` keep the live catalogue in front of you.

APPENDIX C

C

The Polish Loop, on One Page

When you are making your own poster and want the loop without the story, this is it: the sequence that keeps you in prose and spends a render only on a scene that will survive it.



*The authoring loop. You stay in prose; analyze and fix tighten it **before** a render burns time.*

The loop, step by step

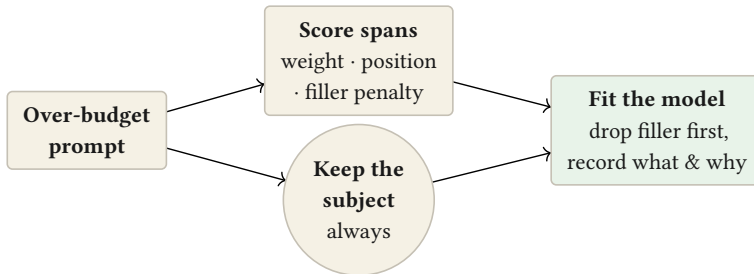
- | | |
|-------------------------------|--|
| 1 · Draft | Write the scene in prose — sentences, components, a weight or two, a tight negative:. |
| 2 · Analyze | <code>compile --analyze</code> . Read the grade and the risks. Aim for LOW RISK. |
| 3 · Fix | <code>compile --analyze --fix</code> applies the safe edits (backup + <code>smysl</code> record). Handle the structural notes yourself. |
| 4 · Re-analyze | Repeat 2–3 until LOW RISK. This is where the time is saved. |
| 5 · Compile | <code>compile --out scenario.hjson</code> . Inspect with <code>--explain</code> / <code>--dry-run</code> if unsure. |
| 6 · Improve (optional) | <code>compile --improve</code> raises the aesthetic score automatically and writes the winner into the scenario. The corpus is its memory — no re-marching. <code>--improve-skip-good</code> leaves already-good scenes alone. |
| 7 · Run | <code>scenario scenario.hjson</code> . Tune seed / steps / guidance / scheduler. |

The commands, in order

● ● ● *A whole polish loop*

```
$ plakat compile prompts.txt --analyze           # will it render?
$ plakat compile prompts.txt --analyze --fix      # trim & repair safely
$ plakat compile prompts.txt --analyze           # confirm LOW RISK
$ plakat compile prompts.txt --out scenario.hjson --smysl
$ plakat compile prompts.txt --improve-all       # (optional) auto-raise
the score,                                     # winner written back
                                              #
to the HJSON
$ plakat scenario scenario.hjson                # spend the render
```

Why analyze first



Model-free budget packing: a trim you can audit, not a silent truncation. The LLM reword is the fallback.

The whole point is arithmetic: a HIGH-RISK scene wastes minutes per render and often several renders before you accept the scene, not the seed, was wrong. `--analyze` turns that into a five-second sentence. Subtract before you add — over-stuffing is the default failure, and the cure is almost always **fewer** subjects.

smysl: the memory of the loop

You never author `smysl`; it records **why**. Born at `--fix`, it accumulates across runs and answers questions later.

<code><name>.smysl</code>	The corpus beside your prose: each fix a claim grounded in the risk it answered, plus budget-pack decisions.
<code>compile --trace "X"</code>	Why is X in — or gone from — the prompt? Reads the corpus; renders nothing.
Accumulates	Repeated <code>--fix</code> merges new decisions with old; identical ones de-duplicate.
Active memory	Under <code>--improve</code> the corpus is a tabu list: every rewrite tried (kept or reverted) is remembered, so the loop never re-marches and can skip already-good scenes.

Reading a feasibility report

8–10 / **LOW RISK**

Good to compile.

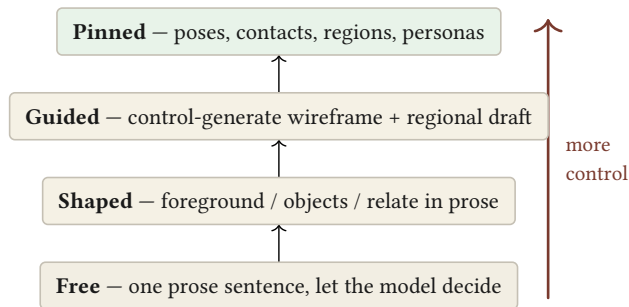
4–7 / **MODERATE**

Fixable — run --fix, then reduce the flagged subjects by hand.

1–3 / **HIGH RISK**

Over-stuffed or impossible for this model. Subtract, split, or climb the control ladder.

The control ladder, when analyze says climb



The control ladder. Start free; climb only as far as the image needs — every rung costs effort.

THE ONE RULE

Stay in prose. Everything — analyze, fix, trace, compile, improve, control-generate, naturalize — serves the prose or the pixels. Climb the ladder only as far as the image needs, spend a render only on a scene that has earned it, and let the corpus remember why you made each choice.

AFTERWORD

About This Book

This book followed one poster because that is how the craft is actually learned — not by reading what every flag does, but by watching an image become itself, and reaching for each tool at the moment it was needed. NIGHT MARKET was never the point. The point was the **order**: install before you generate, write before you render, analyze before you spend, subtract before you add, edit before you re-roll, and finish before you ship. Learn the order and the flags follow.

If there is a single idea to carry out of these pages, it is the one the first chapter opened with and the last one closed on: **you write in prose, and everything else serves the prose or the pixels**. plakat is a large studio of specialised tools — a compiler, a critic, a composer, a retoucher, an enlarger, a signer — and the way to keep them from overwhelming you is to keep them in their places. The prose is yours. The scenario is output. The corpus remembers why. Each render is a bet you place only after the odds look good.

Where to go next

This book is the journey; the reference is the map. Every command here has a `--help` with the full set of flags, and plakat's RFCs document the design behind the bigger features — persona, control-generate, etch, naturalize, texture, comic — when you want the reasoning, not just the lever. Two commands are worth keeping at your fingertips as you strike out on your own poster:

• • • *Your two companions*

```
$ plakat doctor --capability      # what can this machine make?  
$ plakat compile prompts.txt --analyze # will this scene render?
```

The first keeps your ambitions matched to your hardware. The second keeps your evenings spent making instead of waiting. Between them, they are most of the discipline this book tried to teach.

Colophon

Typeset with Typst in Libertinus Serif, with terminal mockups in DejaVu Sans Mono. The screens are faithful to plakat as of plakat 6.30.0 commands are real and current, though a model's exact wording and timings will vary with your hardware and your prose. The running example, the town of the night market and its vendor, is invented — a scaffold for the features, not a place you can visit.

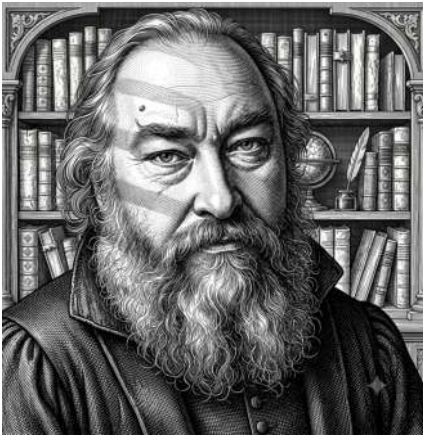


Now go make the one you have been imagining.

Vladimir Ulogov

AFTERWORD

About the author



Vladimir Ulogov.

Vladimir Ulogov has spent decades building infrastructure for distributed systems — the kind of software that watches other software. Early in his career he worked on monitoring and telemetry platforms; later years took him into federated observability, telemetry buses, and the architecture of systems that have to make sense of millions of data points without losing the thread.

Observability, in the end, is a discipline of **coherence** — of never reporting a state the system cannot account for, of insisting that every signal follow from something real. It is not an accident that a studio he built for image-makers carries the same instinct into every picture it helps make: never a render the prose cannot account for, and never a change whose reason is

lost. The analyzer that reads a scene before it costs you, and the small provenance record that remembers why each edit was made, are observability turned toward art.

What makes him slightly unusual in his corner of the industry is a tendency to write his own tools — not small utilities, but programming languages and formats. The Bund language (its compiler, its VM, its document store, its parser) lives in a long series of Rust crates on crates.io. `rust_dynamic`, `rust_multistackvm`, `bundcore` — each is a building block that exists because the off-the-shelf options didn't fit the shape of the work. `plakat` grew the same way: an engine for turning prose into pictures that, along the way, learned to **reason about** the prose — and even the little knowledge format it uses to remember its own decisions, `smysl`, is another of his crates, folded in where a picture needed a memory.

A work of love

`plakat` is open source, under a permissive licence — you can read it, fork it, study it, modify it, and pass it on. Strictly speaking the licence also lets you sell it; the author would, gently but firmly, disagree with your doing that. `plakat` was not designed as a *for sale* project. It is a work of love made for the people who can least afford to pay for software — the designer on a battered laptop, the student making a poster for a club with no budget, the artist who simply wants a tool that runs on their own machine and answers to no one. It carries no analytics, no telemetry, no upsell; the binary will never phone home. Your images are rendered entirely on your own machine, and nothing leaves it unless you deliberately choose to route the optional prompt-reasoning steps through a hosted language model — a choice you control, one `plakat` never makes for you, and one you can decline entirely by keeping those steps on-device or on a local Ollama model.

A note on cooperation

Vladimir believes firmly in the human capacity for mutual help — that we make better work, and live better lives, when we share what we know and what we build. Open source is one of the most concrete expressions of cooperation our era has produced: code read, improved, and passed forward without payment, without

permission, by people who will never meet. If this book helps you make one image you are proud of — a poster, a portrait, a thing you imagined and can now hold — that is enough.

Where to find more

GitHub [@vulogov](#) — the source for `plakat`, `Bund`, `smyzl`, and the dozen-plus Rust crates that carry the infrastructure. Issues and pull requests welcome.

LinkedIn [/in/vladimirulogov](#) — posts on observability, the occasional long-form essay.

YouTube [@vulogov](#) — talks and walkthroughs from the conference trail.

If the tool ever gets in the way of the picture instead of out of it, open an issue on GitHub. The author reads them.